Finding the Net: A Predictive Look at NHL Shooting Performance

Anthony Mollica

Department of Computer Science

Adrian College

Supervisor: Yasser Alginahi

A capstone project proposal submitted to the Department of Computer Science in partial fulfillment of the requirements for the Degree of Arts in Data Analytics at Adrian College.

Fall 2025

Abstract

This project analyzes over 360,000 NHL shot events from the 2022-2025 seasons to identify which factors influence goal-scoring likelihood. Using a combination of statistical testing and Machine Learning methods, this study evaluates numerical and categorical features through ANOVA, chi-squared tests, Logistic Regression, Random Forest, and Decision Tree models. The results show that shot distance and shot location are the most significant numerical predictors of scoring. The most statistically relevant categorical features for scoring are the player name, goalie name, player position, and strength situation. Logistic Regression provides interpretable results, but it struggles with predicting because of the imbalance of the dataset, where only 7% of shots result in a goal. In contrast, models like Random Forest and Decision Tree capture the nonlinear relationships and identify individual player skill as the most significant predictors of goal likelihood. Even the most accurate models struggled to predict goals consistently, showing how rare and random scoring happens in professional hockey. Overall, the findings in this research will highlight both the challenges of analyzing NHL goals and the dynamic, fast-paced game of hockey is a game of skill, strategy, and situational awareness.

Table of Contents

Abstract **1**

1 Introduction 4

2 Literature Review 5

3 Methodology 13

3.1 Data Collection 14

3.2 Data Cleaning and Preprocessing 15

3.3 Data Description and Profiling 16

3.4 Project Timeline 17

4 Classifying Features 17

4.1 Logistic Regression Classification 18

4.2 Random Forest Classification 19

4.3 Decision Tree Classification 20

5 Exploratory Data Analysis 21

6 Statistical Analysis & Predictive Modeling 26

6.1 Formulas 26

6.2 ANOVA F-Test 28

6.3 Chi-Squared Test 28

6.4 Logistic Regression Analysis 30

6.5 Decision Tree Analysis 31

6.6 Random Forest Analysis 32

6.7 XGBoost Analysis 33

6.8 ROC-AUC 34

7 Discussion 37

7.1 Classification Results 37

7.2 Exploratory Analysis Results 38

7.3 Statistical Analysis & Predictive Modeling 39

7.4 Reflections 40

Conclusion **42**

Acknowledgement **43**

References **44**

Appendices 46

1 Introduction

Hockey is a fast-paced team sport that is played on ice where two teams of six players compete to score more goals than each other by shooting a puck into the opponent's net. There are three different roles on a hockey team. Forwards, who consist of wingers that play on the outside and centers play in the middle, are primarily responsible for producing goals and creating offensive plays in the other team's end of the ice. Defensemen have the focus on stopping opponents from scoring on the defensive end of the ice. Lastly, the goalies whose goal is to defend and stop the puck from entering their net. The key performance stats are goals, assists, and points (sum of goals and assists). Plus/minus is the measure of the times a player was on the ice when their team or the opposing team was scoring. Understanding the basic statistics is necessary for understanding shot-level analysis.

The National Hockey League has a long history of performance statistics that capture individual player statistics, team statistics, and salaries across decades. With a new era of sports analytics, NHL teams, players, and front offices use data analytics to help them with team management decisions, evaluate player performance, and evaluate organizational strategic decisions. This project aims to analyze NHL shooting statistics to show trends, look for patterns, and predict player metrics and success.

This project will use NHL datasets from moneypuck.com [1], which includes a list of different shot statistics for each shot that was taken from years 2022 to 2025. The project's focus is on analyzing shot metrics similar to shot location, shot type, game situation, and several others. The prediction is whether a shot that was taken results in a goal or not. By examining over 300,000 shot events, we can make predictive models for shot outcomes.

The analysis will examine the positional differences between skaters and goalies. The goal is to identify key metrics that define the success of shots in the NHL, along with developing an analytical approach that provides scouts, players, and coaches with useful information.

2 Literature Review

Over the past decades, research in the National Hockey League (NHL) has changed from traditional analysis of salary and team data to more advanced models that show the complexity of individual player and team performance. Earlier research shows aspects like player compensation, sales, and the effect of injury in the NHL. However, as game data and player data have grown, modern research has grown to rely more on advanced statistical methods, efficiency models, and Machine Learning methods to understand player value, performance and evaluate overall team performance. Using these studies together can provide a solid foundation for analyzing NHL data in a modern analytics way. The development of NHL data can allow analysts, coaches, and general managers to understand the quantitative and positional point of player performance.

Datasets collected from MoneyPuck [1] provides access to research on individual, team, and shot performance statistics throughout the NHL. These datasets are very important to the project analysis because they cover traditional and advanced hockey performance statistics. The player shot datasets cover the 2022/23, 2023/24, and 2024/25 NHL seasons. I can also analyze many advanced analytics using the individual player and shot data set. Further, these datasets covers multiple NHL seasons, allowing for comparisons between historical and recent player performance. Using a reliable source such as MoneyPuck makes this project credible and reproducible. The following review of the literature can examine the evolution of NHL research, and we can use this reference to make the project research relevant and focused. The project can provide a solid foundation for future research and performance modeling in sports analytics.

It is also important to acknowledge the AI tools that I used for this project. ChatGPT [2] was used to give examples on the structure of my research, as well as formatting tables or images into LaTeX style. Google Gemini [3] was also used as a tool to help me understand code and handle any errors in my analysis.

McLean and Veall [4] offer an early systematic look at salary differentials in the NHL, highlighting how performance translates to financial compensation. Their work is significant to the project because it shows what was before the salary cap era, which provides a baseline for understanding how the market shaped the earnings of professional hockey before modern contract restrictions. Having the ability to analyze stat performances, such as goals, assists, and games played along with the salary, they highlight the performance statistics that are the most important to determining the pay. This research helps illustrate the contract logic to player value at the time and provides good insight into how the star players differ from the role players in the NHL. Further, McLean and Veall's findings point out broader questions about the work NHL players put into their craft, in particular, the performance-based earnings can explain salary differences. As one of the main contributors to hockey salary economics, their study remains useful for how player salaries have evolved over time from pre- and post-salary cap era in the NHL.

Donaldson, Li, and Cusimano [5] provide strong research on the economic burden of injuries in the NHL, showing how the time lost from injuries carries a financial cost for both the player and the teams. Data was analyzed from 2009 to 2012, collecting data from players who missed at least one game due to injury. The study found that 67.1% of

the players missed at least one game from injury, with an average of 8.8 games lost per season, which is close to 11% of the schedule. The injuries cost the teams an estimated 653 million dollars in salaries, while concussions cost an estimated 128.5 million dollars, covering 323 cases. Across the 30-week sample, 870 injuries were analyzed. Most injuries were fewer than 30 games missed, but the ones that were severe made a large financial impact on the teams and insurance companies involved in the NHL.

Lambrix et al. [6] research on goal-based performance metrics that account for the importance of goals scored, offering a more unique measure than simple totals. The study focuses on a variable called Game Points Importance Value (GPIV), which basically looks at how important a goal is to the team's game in different scenarios. The GPIV is based on changes in win/tie probabilities before and after a goal. The analysis showed that GPIV is a higher rank when the goal is scored late in a close scoring game, and early, large leading goals impact on the rating very minimum. They found that players with high GPIV were Ovechkin, Wheeler, and Okposo when they included goals, assists, points, and plus-minus into the equation. In summary, GPIV is a statistical model that was developed to measure important goal scoring to team success. This is very useful for NHL coaches to determine which players to put on the ice in critical moments of the game.

The effort to categorize performance in hockey has evolved tremendously. Riley [7] researched the variety of NHL player performance using structural equation modeling, which is a technique that shows relationships between different performance variables. Riley's study looked at attributes like scoring ability, defensive effectiveness, skating speed, and teamwork to form an overall analysis of individual players' contributions on the ice. Further, with more recent analytic models like GPIV, sports analysts are allowed to measure player statistics, but also their actions and how they contribute to team successes. Overall, Riley's research, which was done a few years ago, shapes how the research is done now; it is continuing to evolve into newer and improved analytics like GPIV.

Pelloneova [8] applied efficiency analysis to determine the technical efficiency of some of the NHL greats, raising questions on how to define "the best player" historically. This work demonstrates a shift toward more technical and well-thought-out methods for understanding some of the on-ice roles. The dataset covers players all the way from 1944 to 2024, including some of the all-time greats like Gordie Howe and Wayne Gretzky. The study found that 69.7% of players were Canadian, a clear large majority of the NHL. Pelloneova used Data Envelopment Analysis (DEA), which is a method used to evaluate technical efficiency in NHL players. Statistical measures like goals, assists, points per game, trophies, and championships weighed heavily into the research. The results showed that Wayne Gretzky was ranked as the most technically efficient player, mainly due to his production, not because of how many games he played. Gretzky holds the record for most assists and points per game in the NHL currently. Alexander Ovechkin just passed Gretzky on goal scoring in the 24/25 NHL season. The other players that were highly efficient were Butch Goring and Serge Savard because of their offensive and defensive abilities. To build on this, Pelloneova uses similar efficiency as Lambrix et al. [6] to measure and analyze performance in the NHL. By being able to compile statistical output with efficiency modeling, modern research is not just about player production; the efficiency models can find players who are producing at the most important times of offense and defense in an NHL game.

As the sports analytics world is on the rise, Machine Learning and data science methods are continuing to be applied to the NHL. Luu et al. [9] demonstrate that Machine Learning models can outperform logistic regression in terms of player injuries across the ten seasons they looked at, examining the prediction of advanced statistics in the NHL. In their study, 2,109 NHL players and 213 goalies were included. Metrics that were looked at included accuracy, area under the receiver operating characteristic curve (AUC-ROC), which measures how well the model performs between two classes, F1 Score, measuring precision and recall of injured players, and Brier Score Loss, which measures the probability of predicting injuries. Players with AUC values between 0.6 and 0.7 were poor, and 0.8 to 0.9 were good. The AUC used a Riemann sum for computation, showing the differences between models. The results came back very accurate. AUC had an accuracy of 94.6% for predicting injuries in the NHL. The goalie injuries model was even more accurate at 96.7% . The study done by Luu et al. shows how Machine Learning is used in sports analytics. The models are very important for coaching decisions, managing players' workloads, and contract negotiations. Using Machine Learning algorithms can make teams and players more proactive with their injury management and provide a better plan for a player's performance.

Moreau, Perera, and Swartz [10] apply Functional Data Analysis (FDA) to determine the value of NHL draft picks, using data-driven examples to assess the value of a long-term player. FDA is critical to determining NHL draft picks using data to model player performance over time. In Moreau et al. study, they use linear regression models that are more advanced than the traditional linear models. The FDA approach captures the time trends and evaluates the player based on early draft picks. B-spline basis functions were used to ensure flexibility, and constraints were used to make more logical decisions. For example, a first-round pick is always expected to be more valuable than a late-round pick. Overall, the techniques that Moreau et al use are a strong evaluation of predictive modeling in NHL and pro sports in general. Sports analysts can use these modeling techniques to find "dark horse" draft picks and avoid players that are more likely to be "busts". Functional analysis is continuing to become more relevant for understanding team flaws and being able to make better, weighted decisions.

J. Wu et al. [11] use performance metrics to analyze how goals and positive attributes contribute to winning games in the NHL. The dataset was cleaned to form a 25-game period of analysis. The main performance indicators looked at were related to player advantage (power play), penalties, and faceoffs. The data was then used for cluster methods that were categorized into three different types: unbalanced winning (or goal difference), balanced, and unbalanced losing match. There was also a study of five key determinants of match performance that included maintaining puck possession, ability of shot defense, aggressiveness performance, unfavorable state (without the puck), and major penalties. The study revealed that the teams with the higher ranking often shared a stronger unbalanced winning streak, meaning they were winning by larger differences. Overall, the analysis gives a strong evaluation of NHL team performance, and it can help this project in looking at similar statistics.

G. Pischedda [12] runs the study out of Ottawa, Canada, using a Machine Learning approach to analyze NHL data in 2013 that covers around 517 games. The two main datasets included are Win/Loss and Home Wins/Away Wins. Pischedda found that CluseteR, VH-A, and Cat+Vars gave the highest accuracy for analyzing the variables. There

was a total of 18 models built, such as Decision Trees, Artificial Neural Networks(ANN), and Clustering. Further, the methods used to analyze the data worked decently for predicting NHL games with an accuracy of 60% .

"Blowouts" in hockey refer to a team beating another team by 6 or more goals. Chachad et al. [13] analyze the impact of what it does on game performance in the NHL. Data was collected from 2005 all the way up to 2019 using box scores and regular-season game statistics. This accounted for 16,792 games. Multiple regressions were used with the blowout game goal differential as the predictor. A binary model predicted game outcomes, and a linear model predicted goal differentials. The models also looked at the location of the game, if the game was back-to-back, the schedule, and the NHL team winning percentages. They found out that the average number of goals scored in a blowout game was 6.44 and found that the total number of blowout games was 286. After running the correlation in the models, there was no clear relationship between blowout games and how the teams performed in the next game (for both winning and losing teams).

O. Siivonen [14] makes a bachelor's thesis on predicting player performance in the NHL from 2022-2024. The models used the number of goals a player will score as the predictor. The model used was predicted well with an average error of 0.171, and 85% of the players' goals were predicted within three goals. The R-squared for the model was approximately 0.83 for both training and testing, meaning that the model explained the goal variation well. The model was good at predicting average goal scorers within 10-13 goals, but not as well for predicting high goal scorers, as it underestimated lower amounts of goals scored. Overall, the regression model that was used was a strong predictor for predicting individual player goals scored, except for the outlier of players who score more frequently.

J. Viljakainen [15] also wrote a Bachelor's thesis, but analyzed the ability to measure the important values that contribute to NHL success in the regular season. The data was used from 21 different files compiled from 2010 to 2025. Viljakainen used MATLAB to perform the main analysis and computed the correlation between team statistics and points using the corrcoef function. Linear regression was also used to see the development of each stat over time. After analyzing the results, we found that wins, regular wins, and regulation overtime wins had the strongest positive correlations with losses, with losses being strongly negatively correlated, as you could guess. There was also a prediction that 2025 to 2027 would have increasing trends in most statistics that were analyzed, besides a few, like losses and win percentage in three-goal games. Cross-validation was also performed with the 2024-2025 season, showing that the model predicted well overall, with the average error around 12.8%. In general, the analysis was nothing over the top, but it was very focused on the main statistics in the NHL and stayed consistent.

Pitassi et al. [16] use Artificial Intelligence models to explain advanced NHL analytics. The study used Machine Learning to produce analytics related to coaches, players, and scouts throughout the NHL. They found that the key predictors of success are offensive awareness, puck control, and faceoffs. "Partial Dependence Plots" (or pdp) and "gradient boosting" were used for feature importance and show how the models have been elevated from basic analysis. In the final phase of their analysis, they used the "Random Forest" model to look at cross-validation, and it determined there was strong stability but room for improvement because the analysis had similar and narrow predictors. Overall, the model was consistent and gave good player performance insights, but it lacked diversity,

meaning it was too focused.

The regression analysis that DiBenedetto [17] does highlight how important the contract year influencing NHL player performances using offensive and defensive metrics. The results showed that the players in contract years tend to decline in their individual performance. In the total points model that DiBenedetto uses, the coefficient was -7.38, meaning that the players, overall, on average, produce seven fewer points in the contract years than they normally do in non-contract years. The goals model was -2.54 on average, and the assists model was -4.84 on average. Together, this shows that player offensive production decreases in general. Similarly, the blocked shots model points out that the coefficient in their contract year was -5.21, meaning that the players' defensive side of the game blocking shots goes down. This could indicate that the players are trying to avoid injuries ahead of new contract negotiations. Although the hitting model said that there was little to no change from contract-year hits to non-contract-year hits. Across all the models, games played, and team points were consistent predictors for individual performance, meaning that opportunity plays a large role in determining player outcomes. The defensive variable turned out to be significant in most of the regression models. Age and year were insignificant in predicting the model, meaning that experience doesn't affect contract year performance. The results together give a strong argument that contract-year performance is stressful for the individual NHL player as they try to prove themselves in every way possible. Overall, the players take fewer risks so they can avoid injury, and the players' performance goes down on average a little because of the stress it puts on the. This reference will help me understand the challenges NHL players go through during the uncertainty of a contract, and help me look at how you can use regression models to look at NHL analytics.

The study Liu et al. [18] analyzed NHL players and performance in the NHL using deep reinforcement learning (DRL) and regression trees. The data set being used records over 3.3 million on-ice plays during the 2015/16 NHL season. The features that the dataset used are puck position, velocity, time remaining, power play, score differential, and action outcomes. The DRL frame represents each team as a Q-function that estimates the probability teams will score the next goal given specifics like game state and action. Liu et al. use temporal difference learning to associate plays and positions on the ice to look at scoring probabilities. They found that the shot success probability depends strongly on the distance and the angle of the goal. Goal Impact Metric was used to measure how each action effects a team's expected scoring probability. The metric values correlate strongly with statistics like goals, assists, time on ice, and salary. The mimic regression trees were used to repeat DRL functions and impact scores. The key features the model found were goal occurrence, shot on goal, success, puck location, and puck velocity. The findings show how DRL and Machine Learning can understand player performance and what influences scoring outcomes. Further, the study represents a great step towards how hockey analytics is progressing for AI modeling player performance in sports. This study can help by giving different types of new Machine Learning methods to look at when evaluating player statistics in large datasets.

Johansson et al. [19] use Corsi, which is a puck measurement that counts all shot attempts like shots on goals, missed, or blocked shots. It is labeled as Corsi For, Corsi Against, and CF percentage; all of these give a team's total shot attempts. Typically, the higher the CF percentage is, the better the team is in the playoffs. They found that before 2016,

it was common for teams with high CF percentage to make it to the final rounds and even win the Stanley Cup. Over time, NHL teams are starting to generate more shot attempts. The one thing that Corsi doesn't measure is the shot quality. The analysis Johansson et al. do focus on analyzing the 'expected goals', that is, the likelihood a shot results in a goal being scored. They found that since 2015, the expected goals stat is more helpful at determining playoff success than Corsi. So now the teams are focused on shooting the puck more and trying to get to better scoring areas on the ice to increase their chance of scoring. Overall, this study can help understand the trends of how teams are using some of the basic team stats to give themselves a better chance of winning games. This study will help the project by understanding how more goals are scored from a team perspective.

The study done by Farlinger et al. [20] analyzes how off-ice performance can relate to on-ice skating ability in male hockey players. The study showed thirty-six players from ages fifteen to twenty-two with an average of 10 years of experience in hockey. The skill level of the players was not NHL, but just average hockey players who played in Bantam or Varsity level. The testing was done over a three-day period. The on-ice testing was a 35-meter straight line sprint, a cornering S-shape test that measured speed and agility. The off-ice tests consisted of three different types of jumps, anaerobic test, sprints, shuffles, push-ups, and a shuttle run. These tests aimed to test the leg power from all directions, agility, core stability, and overall strength. Farlinger et al. used correlation and multiple regression to find a relationship between on-ice and off-ice performances. They found that there was a strong correlation, looking at the 30-minute spring and the hop jump distance compared to on ice sprint performance, the R-squared was 78%. The other tests off ice were moderate correlations and did not show significance to the playing level or on ice skating ability. Overall, the study that was done was very thought-out and was more of an older approach to how they tested on ice performance. A lot has changed since 2007, and the testing approaches that we have seen above are way more in-depth than they used to be.

Gaudiani et al. [21] analyze NHL players who have gone through hip arthroscopy or labral tear from 2003 to 2023. They left out the players who did not have surgery, or if they had other conditions going on that were related to their hip. A 2 to 1 matched control cohort was created based on the player's position, experience, age, performance level, and BMI. Data like demographics, ice time, and performance data were collected from one to three seasons before and after the surgery was conducted. Metrics that were included were all basic individual player statistics; the study also included salary data to analyze. Out of 75 players that were included, 88% returned to play after the hip arthroscopy. They found that their performance slightly declined in the first season after one season post-surgery, with Corsi and goals reduced slightly. However, three seasons after the surgery had happened, the performance showed that it was not significantly different than before their hip problems. Gaudiani et al. found that the players' earnings and career longevity were not significantly affected. The players who didn't make as much earned a little less post-surgery, and the players who made more earned about the same as before. Overall, the research shows that hip surgery takes a few years until the player feels back to normal, and the salary remains close to the same. The study compares to seven of the other references used so far and shows that regression can be used very well when tracking analytics over years.

Goal scoring analytics research paper by Parnican et al. [**?**] will help me use the shot data set found for my research project. The study analyzes 511 different even-strength goals that were randomly selected from 129 different games in the 2020-21 NHL regular season. The goal of this study was to look at how puck possession, game situation, and number of passes influence goal scoring. Parnican et al. used one-way ANOVA and Tukey HSD post hoc tests to see if there were significant differences between the variables that they used. They found that goals were more frequent from puck possession that was gained in the offensive zone (in other words, a turnover from the other team in their zone). This had the highest mean score out of the variables, which emphasized that winning battles in the offensive zone is important for scoring goals. They found that the offensive zone play generated more goals than rush attack or transitional goals. Further, puck movement that had one or two passes before shooting the puck was most effective for scoring goals. Overall, this study was done simply but effectively. It shows how important it is to manage the puck in all zones of the ice, especially in the offensive zone, where a goal can happen at any given moment. This goal scoring analytics can help teams find better methods for scoring and focus on specific areas on the ice in helping the players visualize goal scoring.

To build off [**?**], another goal scoring analysis specifically conducted on scoring rates will be used from Buttrey et al. [22]. The analysis was used from looking at how offensive and defensive teams are, the home-ice advantage, and the power-play situation to determine scoring outcomes. The study shows a log-linear Poisson model that shows the expected goal rate as the output to four different factors: league-wide base rate, team specifics such as offensive and defensive coefficients, and home versus away. Instead of using a large range of parameters, Buttrey et al. use only 60 parameters for the goal rate to keep the analysis more concise and interpretable. The next thing that they analyzed was box-score data from the 2008-09 NHL season. An S-Plus script was used to change the data into detailed "intervals" like 5-on-5, 5-on-4, and 4-on-5 game play. This part of the project used 10,000 different intervals. The results that they got showed that goal scoring is correlated with the Poisson process model. Power-play increased the scoring rates by 175% compared to even strength, and shorthanded situations reduced scoring to one-third compared to even strength. Next, Buttrey et al. fit a generalized linear model and found that team-level offense and defense coefficients were consistent with goal totals. Detroit's offense was 13% higher than Anaheim's, and Boston's was 16.3 percent lower than Anaheim's rate. The model also found that home-ice advantage had an 8.8% higher scoring rate than away. The last thing that was incorporated was a Skellam distribution that estimated win probabilities among any two teams. This gave a good explanation of how offensive and defensive strengths increase winning outcomes. Finally, the data analysis was done very thoroughly and used different comparisons to give meaning to the work that they found.

Another scoring model that will help give ideas for this project, which was done in 2012, used two regression models to predict scoring rates in the NHL [23]. The two regression models that were used is ordinary least squares (OLS) and ridge regression. The statistics that were used included goals, shots, missed shots, blocked shots, hits, faceoffs, and advanced stats like Fenwick and Corsi. The data was used over the seasons 2007 to 2011. Macdonald split each season into odd and even games to account for roster changes, injuries, or trades. Odd-numbered games were used for predicting goal

scoring in the even-numbered games. The models were trained from the 2007-2010 seasons and tested on the 2010-11 season, with the mean squared error being a primary metric. Macdonald used an OLS model to get five predictors that included: goals, shots, hits, hits against, and faceoffs. Both models (OLS and ridge regression) worked better than traditional statistical models because they were based primarily on goals, shots, Fenwick, or Corsi. The correlation between predicted versus actual goals was 0.67, and the lowest mean squared error was 0.053. Ridge regression performed a little bit better than OLS did when predictor variables were correlated. Macdonald also proves that the model's performance improved when they accounted for outlier teams like Ottawa and Carolina, meaning that their goals scored were either lower or higher than expected goals scored. Further, Macdonald provided more analysis on how in-game statistics relate to scoring. Macdonald found that the variable hits against had a positive coefficient, and hits were negative. This meant that teams that were getting hit more were scoring more goals. When involving variables like hits, possession indicators, and faceoffs, the model was more accurate than the original shot-based metrics that were used. In closing, this research will help influence the project by potentially researching expected goals for more recent analytics.

Moustakas and Muller-Schoell [24] use team data to analyze diversity and team performance in the NHL from seasons 2007 to 2011. They wanted to understand how players' nationality, earnings, and team roster can affect the success of teams during the regular season and in the playoffs. The sample took 120 teams over their seasons. Moustakas and Muller-Schoell analyzed regular-season performance by looking at metrics that included wins, points, goals for, and goals against. The playoff performance analytics were evaluated by participation, playoff wins, and progress through postseason rounds, with 0 meaning they did not make the playoffs and 5 meaning they won the Stanley Cup. The control variables were team payroll and the strength of opponents, which accounted for financial and competitiveness across the different NHL teams. Every player who played at least one game for the team was counted towards the team's composition; this was a method to account for trade, injuries, or limited ice time. Moustakas and Muller-Schoell also measured diversity: Richness represented the total number of nationalities on a team, the Gini-Simpson Index represented the probability that two randomly selected players were of different nationalities, and Shannon's Entropy measured the overall diversity within a team. The research included correlation tests and regression tests to measure how diversity was related to regular-season and playoff outcomes. The results showed that there was no significance for the relationship between diversity and team performance across all the models. Team payroll was one of the stronger predictors of success for both the regular season and playoffs. The results showed that financial resources played a stronger role in determining team success during the years 2007 to 2011. In summary, the methodology of diversity with performance gave the idea that social variables are not as much of a factor in winning games compared to team statistics. The diversity may influence locker room dynamics, but in this study, there was no statistical impact on success. This research shows that adding unnecessary factors to a study may not be helpful, and it teaches that not every prediction is correct. Methods for finding team success will be helpful to research similar ideas in this project.

The study done by Chan et al. [25] quantifies how different NHL players contribute to team performance. The data was collected from 2005 to 2010. Chan et al. used k-

means clustering and regression analysis to focus on four forward types, four defenseman types, and three goalie types. The forward types: Top Line, Second Line, Defense, and Physical. The defenseman types: Offensive, Defensive, Average, Physical. The goalie types: Elite, Average, and Bottom. After the classification, a multiple linear regression was used to try and link team success to the composition of player types for each NHL team. The results showed that goalies have the biggest impact on team performance, followed by forwards, then defensemen. The study also found that lower-cost player types like defensive forwards were efficient investments when comparing salary cap data with ice time. Further, the research came up with a trade evaluation tool that general managers could use to predict how successful a team could be if making roster changes. In summary, the study done gives a good example in how players are valued in the NHL when looking at team success. This research will help the project by giving a good idea on how to classify data.

Overall, the literature review examined a range of studies that analyze NHL performance from individual, team, and goal-scoring methods. The evolution of NHL analytics has gone a long way from traditional analytics to more advanced performance modeling. Early studies that McLean and Veall [4] conducted provided insights into how salary and performance were based on before the salary cap era started. While articles like Chan et al [25] deepened our understanding of player contribution to team success by using clustering and regression. Studies like Riley [7], Siivonen [14], and Pelloneova [8] give individual analytics by making efficiency models to evaluate the skill and basic stats to determine their performance. Wu et al. [11] and Moustakas and Muller-Schoellz [24] give a team analytic approach by linking statistics along with team structure to find success. While the goal scoring analytics studies included Parnican et al. [**?**], Buttry et al. [22], and Macdonald [23], gave more in-depth research to show the importance of goals in different game situations and identified predictive variables that contribute to higher goal scoring chances. Together, these studies give a framework for how modern NHL analytics are used. The combination of different research directly supports the goals of my project by showing how to use statistical approaches to evaluate performance, give value to the team or player, and provide data analysis to professional hockey.

3 Methodology

The purpose of the study is to build predictive models to estimate the probability that a shot attempt in a hockey game results in a goal, otherwise known as the expected goals (xG) model. A multi-stage data analytics approach was used to examine how different shot types in different game situations influence the likelihood of a goal being scored. The datasets in this study were used, gathered from MoneyPuck.com [1], which is a public hockey analytics platform that gives play-by-play data from the National Hockey League (NHL).

The methodology contains several different stages. The first stage begins with data collection and profiling, which gives information on the dataset's size, structure, and key variables. Next, a comprehensive data cleaning and preprocessing phase was done to ensure there is consistency and readability for the analysis. This included removing incomplete or inconsistent records, handling missing values, and encoding categorical

variables (shot type, game situation, and shooter/goalie identity), and then standardizing numerical variables (shot distance and angle).

Following data preprocessing, the project moves into feature classification. In this stage, numerical and categorical features are ranked by different models (Logistic Regression, Decision Tree and Random Forest) on which variable is stronger in relation to our predictor variable 'goal'. This step is an important foundation for the project as it determines which predictors are most important to look at for our data analysis.

After feature classification, the research shifts to the Exploratory Data Analysis (EDA), which gives visualizations and identifies key patterns within the dataset. This stage involves visualizing shot distributions, examining scoring trends, and exploring relationships between key features and goal outcomes. EDA provides an initial understanding of how goals occur, where the shots are mostly coming from, which shot types are more common, and how player and goalie skill can differentiate the scoring outcome. These insights help validate the classified variables and develop our expectations for the model-building stage.

Next, statistical testing was done to quantitatively assess which variables had significant relationships with goal outcomes. ANOVA was used to evaluate the numerical predictors, and a Chi-squared test was performed to identify key categorical relationships to goal scoring. These tests confirm which features are statistically significant for our predictive models.

After the most relevant features were identified, the study moved to model development. Four binary classification models were used: Logistic Regression, Decision Tree, Random Forest, and XGBoost. Each model was trained at 25% and tested at 75% on the cleaned dataset. Goal outcomes are coded as 1, and non-goals are coded as 0. Performance was evaluated using accuracy, precision, recall, and ROC-AUC to compare the effectiveness of each model.

The methodology concludes with results and discussion, assessing each model's strength, limitations, and interpretability. Figure 1 presents a visual overview of the workflow, illustrating how each stage builds on the previous to create a structured and clear xG model process.

3.1 Data Collection

As mentioned above, data were collected from MoneyPuck.com [1], which is a public hockey analytics platform that provides historical play-by-play shot data, including goals, missed shots, and goals scored. For this research, shot data from the 2022-23, 2023-24, and 2024-25 NHL seasons were collected to include a large and diverse sample of shot attempts across all the teams, players, and game situations over the past few years. Each shot tracked includes information describing the circumstances of the shot.

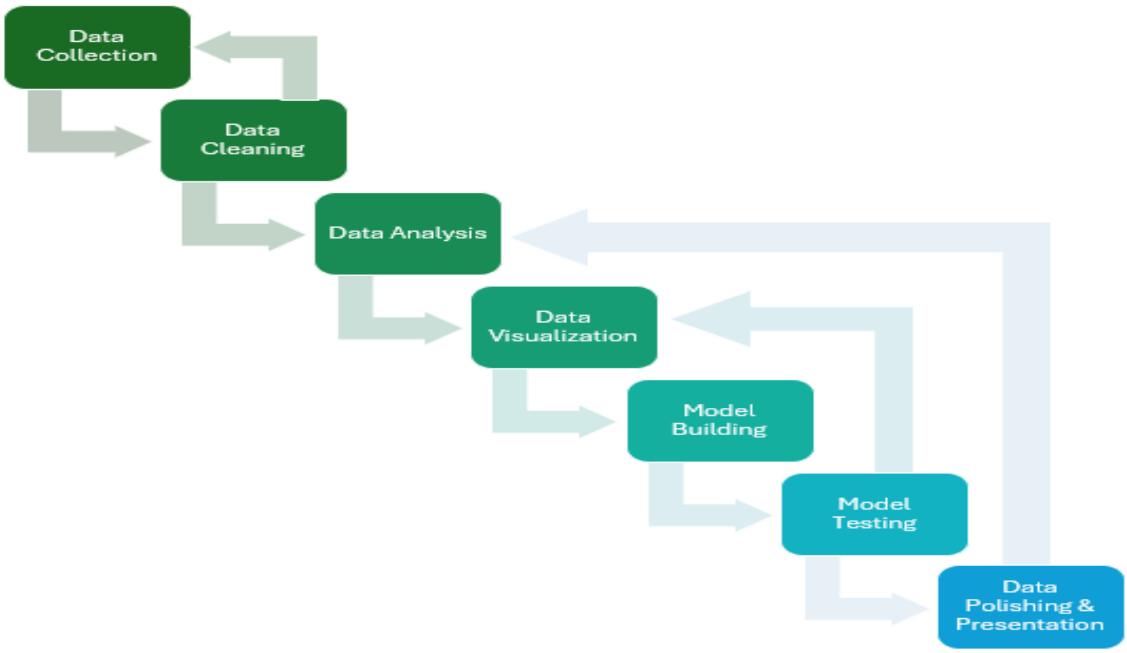

Figure 1: Pipeline Process

3.2 Data Cleaning and Preprocessing

After the collection of the play-by-play shot data from [1], there was a decent amount of cleaning and preparation that was performed to make sure the data is ready to be analyzed. The complete code for this project is provided in the Appendices (A-F). The first step to cleaning was combining the three datasets together using pandas 'concat' function. After that, I had a single data set that was 364368 rows and 137 columns. I needed to simplify the columns to the most valuable features that were needed for the project so I kept 19 of the columns initially, then I created a new feature called 'strengthSituation', which combined data from 'homeSkatersOnIce' and 'awaySkatersOnIce' into one column. Next, I applied the new variable 'strengthSituation' to my list of kept columns and dropped 'homeSkatersOnIce' and 'awaySkatersOnIce'. The cleaning part continued as I cleaned the numerical features so that the missing cells were filled with the median for that column. Similarly, the categorical features that had missing cells were filled with 'unknown'. This way, there were no dropped rows that could lead to skewed data. The final step, I converted the feature 'isHomeTeam' from a boolean to an int so that every numerical column is in the integer format. Overall, the final data set does not have any null values in it, and there were no dropped rows. The final dataset shape for my cleaned file is 364368 rows and 18 columns. The data set was saved and ready to be analyzed for the purpose of the project. The final cleaned variables used for this analysis is provided in Table 1.

Before analyzing the data, I had to preprocess the data so that the data is encoded and standardized, so that the variables are all consistent with each other. Once the pre-cleaned shot data set was imported, I defined my features (or predictors) and target variable (outcome). For this project, I used the variable 'goal' as the target variable, and all the other 17 variables were used for predicting whether it is a goal or not. The next step is to encode the categorical variables because, for classification, we need our models

to have integers. This allows the model to find relationships with categorical variables and goal probability. After encoding, I standardized the numerical features so the data wouldn't skew the results. The last part of preprocessing was to split the data into a training set that covers 80% of the data, and a testing set that covers 20% of the data. The 'random state' was set to 42 to make results more reproducible. This helps give the models a fair assessment and gives better accuracy. After all the preprocessing was finished, I was able to start classifying my models.

Table 1: Final cleaned variables used for analysis.

Variable	**Description**
goal	Indicates whether the shot resulted in a goal (1 = Yes, 0 = No)
arenaAdjustedShotDistance	Distance of the shot from the goal (feet)
arenaAdjustedXCord	Adjusted x-coordinate of the shot
arenaAdjustedYCord	Adjusted y-coordinate of the shot
isHomeTeam	1 if the shooting team was at home, 0 otherwise
goalieNameForShot	Goalie facing the shot
homeTeamCode	Abbreviation of the home team
lastEventCategory	Type of event preceding the shot
location	Zone where the shot occurred (Offensive, Neutral, Defensive)
period	Period of play (1–3, OT)
season	NHL season in which the event occurred
playerPositionThatDidEvent	Position of the player who took the shot
shotType	Type of shot (e.g., Wrist, Slap, Backhand)
shotAngle	Angle of the shot relative to the net
shooterName	Name of the player who took the shot
team	Team of the shooting player
shotDistance	Unadjusted distance of the shot
strengthSituation	On-ice situation (Even Strength, Power Play, Penalty Kill)

3.3 Data Description and Profiling

Each row in the cleaned data set represents an individual shot attempt from NHL games between the 2022-23, 203-24, and 24-25 seasons. The features contain a wide range of situational, distance, and player-specific information that can describe the probability of a shot resulting in a goal. The numerical features such as 'arenaAdjustedShotDistance', 'arenaAdjustedXCord', 'arenaAdjustedYCord', 'shotAngle', and 'shotDistance' describe the mathematical and positional characteristics of each shot taken. Numerical variables such 'as "period' gives probability and insight into the timing and likelihood of a goal being scored.

The categorical features such as 'goalieNameForShot', 'homeTeamCode', 'lastEventCategory', 'location', 'playerPositionThatDidEvent', 'shotType', 'shooterName', 'team', and 'strengthSituation'. These features describe the players, teams, and in-game context that happened for each shot attempt. The feature-engineered variable 'strengthSituation' de-

termines if there were different man-advantage situations, such as even strength, power play, or penalty kill.

Overall, this dataset, which was clean, gives a comprehensive view of every shot attempt recorded across three NHL seasons, giving a balance of numerical and categorical variables that are needed for developing predictive models. Since the data set doesn't have any missing values in it, this allows for statistical analysis and classification techniques to find patterns that separate a successful and unsuccessful shot attempt.

3.4 Project Timeline

Figure 2 shows the timeline the project was produced in.

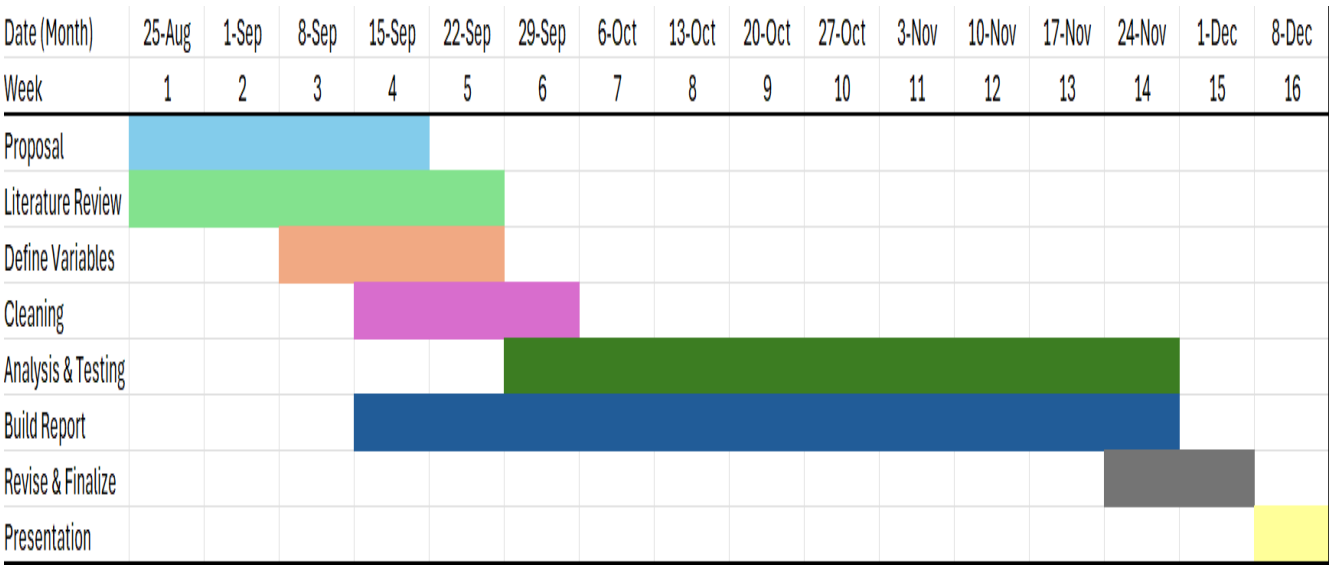

Figure 2: Timeline of Project

4 Classifying Features

After preprocessing the shot data, three classification models were used to predict if a goal was scored or not. Starting with Logistic Regression, a baseline statistical model that uses log-odds to estimate the probability of a goal. Logistic Regression is a classification model that can predict the probability an event occurs. In our case, it can predict whether a shot becomes a goal (1) or non-goal (0); this is known as a binary outcome. Random Forest was used to capture nonlinear relationships that Logistic regression might miss. It works by using a group of decision trees that predict independently, and the majority vote of the trees gives the final prediction. Finally, a Decision Tree Classifier was used to classify the most important features. Decision Tree works by splitting the data based on the features that best separate the classes. It keeps splitting until it reaches a "leaf", which is a final decision that represents the prediction. Overall, Logistic is very interpretable but does not handle nonlinearity, whereas Random Forest and Decision Trees both can handle

nonlinearity and are slightly harder to interpret. See Figure 3 for a visual explanation of the different models.

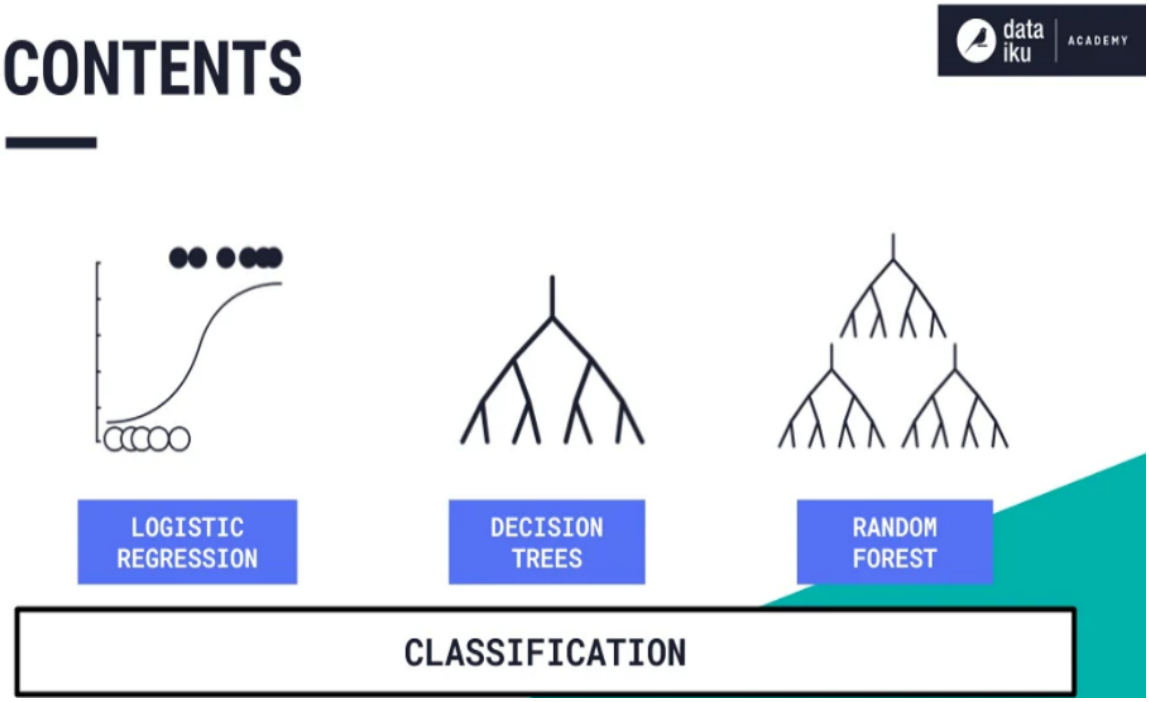

Figure 3: Difference between Classification Models [26]

4.1 Logistic Regression Classification

Logistic Regression is a baseline statistical classification model that can calculate the probability of an event by using a weighted combination of the input features. For the shot dataset, the model will analyze variables similar to shot distance, shot angle, period, location, player position, and team context. Next, the model converts them into a score using a linear equation. The score will then be converted into a single score using a linear equation and transformed into a probability between 0 and 1. Logistic Regression is very useful because it does not take long to train, is simple, interpretable, and effective with more linear relationships of features. On the other hand, it can struggle with complex or non-linear interactions that are common in a hockey data set similar to mine. This is why it was important to use multiple models so I can cover the complexity using different methods.

The Logistic Regression coefficient gives the change in log odds of scoring for a one-unit increase in the predictor. The model classifies 'shotDistance' as the most influential predictor of goal probability in Table 2. The closer the shot is to the net, the better the chance it results in a goal. Variables like 'location' and 'team' are also influential to the goal being scored, indicating that certain locations and different team offenses can affect scoring chance. Variables like 'strengthSituation' and 'shooterName' had smaller effects on goal being scored or not, meaning there weren't as many trends in goals being produced. These results from logistic regression are close to what is expected when

looking at the primary dynamics of goals being scored.

Table 2: Top 10 Features from Logistic Regression

Feature	**Importance**
shotDistance	0.785536
location	0.296425
team	0.181155
goalieNameForShot	0.103822
period	0.101089
isHomeTeam	0.080753
lastEventCategory	0.024068
playerPositionThatDidEvent	0.018097
strengthSituation	0.016905
shooterName	0.013809

4.2 Random Forest Classification

Random Forest is an ensemble Machine Learning method that builds a large collection of Decision Trees and averages them for a more accurate and stable result. For my expected-goals dataset, each tree in the forest learns patterns from different subsets of shots and different subsets of features like shot distance, shot angle, shooter position, strength situation, team, and game context. Every tree can see a subtly different view of the data, the model can capture a wide range of relationships, and it reduces the chance of overfitting because of noise. When the model predicts the probability that a shot becomes a goal, it averages the predictions across all the trees. This gives a more reliable estimate than any single tree could do. Random Forest is effective for shot data, because many features interact in different non-linear ways when it comes to goal scoring.

Using the Random Forest classifier, the number for each feature indicates how well the feature works to reduce error when making predictions from all the trees in the forest in Figure 4. The model identified 'shooterName' as the best predictor, indicating that the individual player who shoots the puck in the net is good at predicting a goal scored. The predictor 'goalieNameForShot' is just below, which means that the goalie being shot depends on the scoring rate. Random Forest identified 'shotDistance' as less of a predictor compared to the Logistic Regression model, but it still made the top 3 in Random Forest. Overall, Random Forest did a better job of classifying individual features due to its ability to capture nonlinear interactions.

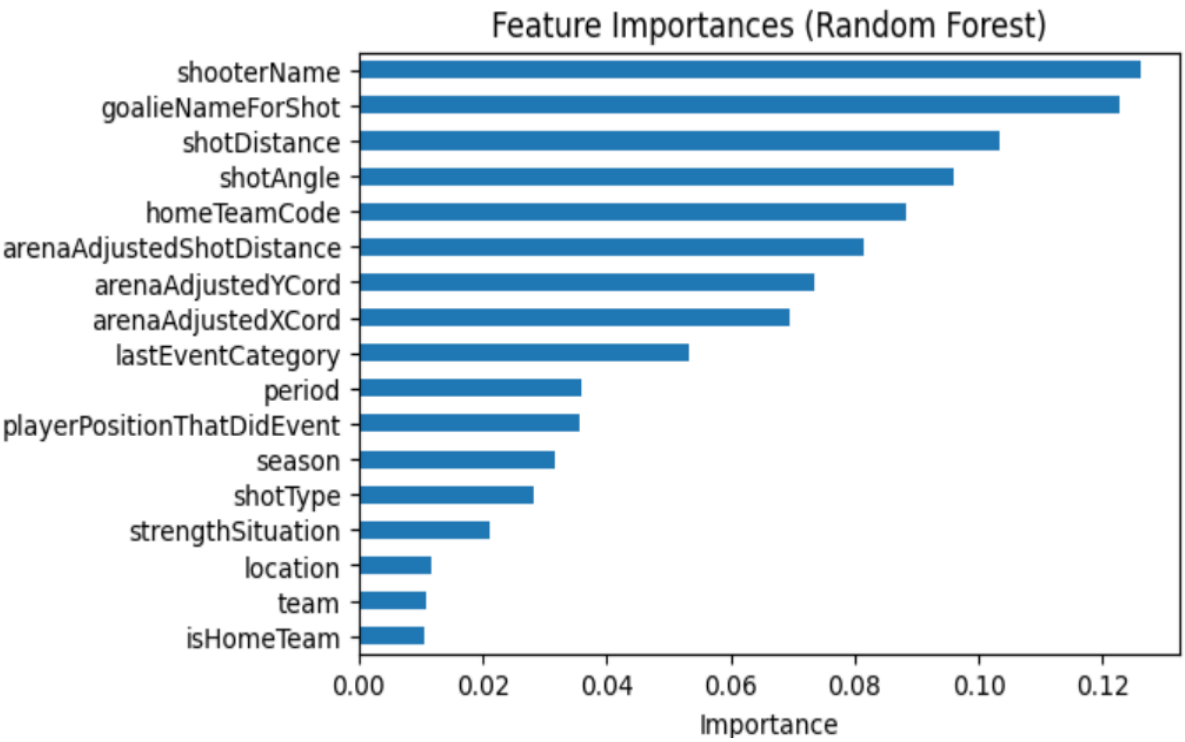

Figure 4: Bar Chart for Random Forest Classification

4.3 Decision Tree Classification

A Decision Tree is a simple and interpretable Machine Learning model that can predict outcomes by splitting the data into different branches based on the most important features. For my shot dataset, the tree learns rules based on: if the shot distance is small, if the shot angle is good, if the team is on a power play, and more. This step-by-step process can estimate the probability that a shot results in a goal. Each decision this model makes is based on reducing uncertainty in our data; in other words, the tree divides the shot into increasingly similar groups where the scoring likelihood becomes clearer. However, even though Decision Trees are easy to understand, they can struggle with complex relationships, which is why they get outperformed by Random Forest and XGBoost.

Using the Decision Tree classifier, the model produced a top ten ranked list for which variable does well with predicting a goal, see Figure 5. 'shooterName' and 'goalieNameForShot' rank as the most influential predictors at around 15 and 14% importance. This indicates that the person shooting the puck and the goalie trying to save the shot matter for predicting goals. 'shotDistance' and 'shotAngle' are also major predictors; the closer the shot and the better the angle can increase the scoring odds of a shot. In comparison to the other classifier models, Decision Tree and Random Forest were very similar at identifying top features; the top 5 were ranked in the same order. In summary, the Decision Tree and Random Forest classifiers are very similar models, and this is evident in the output, which shows that Logistic Regression employs a different algorithm that ranks the features slightly differently, as expected.

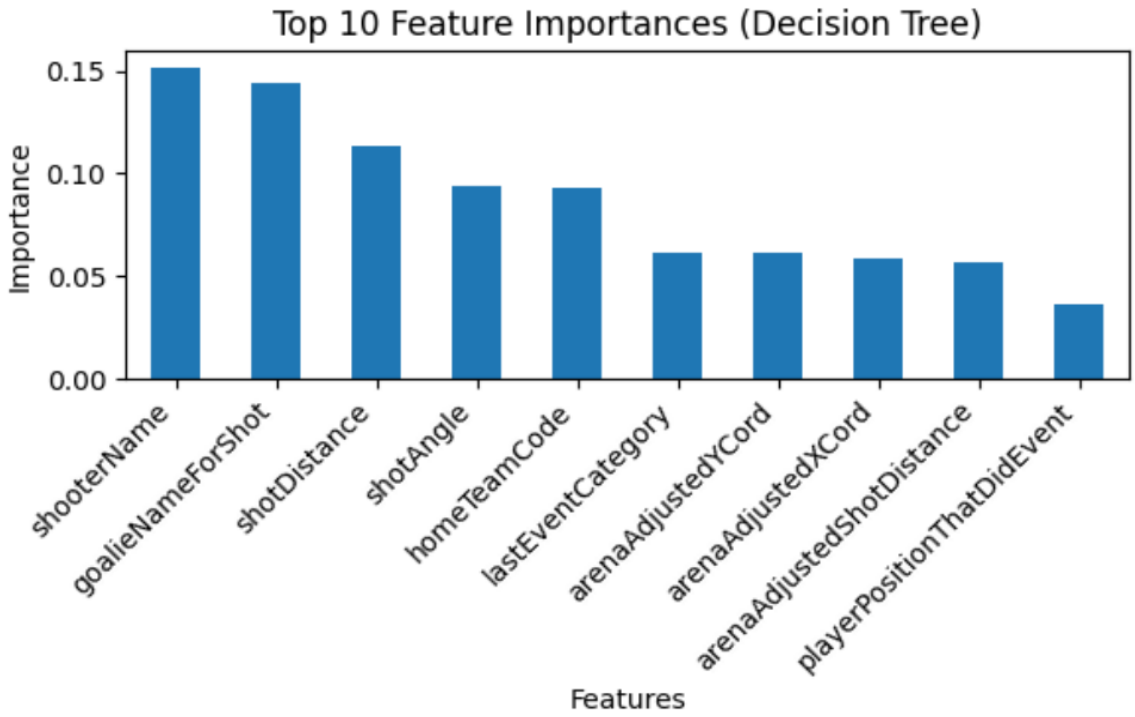

Figure 5: Bar Chart for Decision Tree Classification

5 Exploratory Data Analysis

The next section of the analysis is more towards exploring different features. Since I am working with a shot data set I wanted to research more on what separates a shot from a goal and a non-goal. Analyzing different metrics helps give more variety to the research project and provides interesting results.

The describe function gives the descriptive statistics of the numerical features that each contain 364,368 shots (or rows). The goal feature is a binary outcome (0 or 1); out of the 364,368 shots, only 7% of them resulted in a goal. I found that on average, the shot distance is about 34.46 feet from the net. This means that most of the shots are shot within the blue line and the goal line of the offensive zone. The variables 'arenaAdjustedXcord', 'arenaAdjustedYcord', and 'shotAngle' indicate that shots are generated equally from the same angles on both sides of the rink since the mean is around 0 for all three. It appears that most of the shots were generated within the first two periods, with 'period' having a mean of 2.02. The shot angle ranges from -88 to 88, meaning that goalies are receiving shots from both the left side and right side of the net equally. There are slightly more shots produced by the home team, as the 'isHomeTeam' feature average is 0.51. The shot distances average is 33.6 feet, showing that there is a mix of close and long range shot attempts given, the max is 98.41 feet and the minimum is 1 foot.

Overall, the data shows a wide range of shot locations and distances in Table 3. There are lots of clustered shots near the center of the ice and in front of the net. Out of the 364,368 shots taken, only roughly 25,870 of them resulted in a goal. The descriptive

statistics help give basic but important detail to how each shot attempt is measured in the NHL.

Table 3: Summary statistics for numerical shot data.

	goal	arenaAdjustedShotDistance	arenaAdjustedXCord	arenaAdjustedYCord	isHomeTeam	period	shotAngle	shotDistance
count	364,368	364,368	364,368	364,368	364,368	364,368	364,368	364,368
mean	0.0708	34.4600	-0.2997	-0.2363	0.5120	2.0298	0.0202	33.5969
std	0.2564	19.8276	64.1151	20.4895	0.4999	0.8404	39.8485	19.5944
min	0.0	1.0	-100.0	-55.0	0.0	1.0	-88.4518	1.0
25%	0.0	17.4642	-64.0	-16.0	0.0	1.0	-31.6075	16.4924
50%	0.0	33.0	-2.0	0.0	1.0	2.0	0.0	32.0156
75%	0.0	48.8262	64.0	15.0	1.0	3.0	31.7595	47.6760
max	1.0	99.0	100.0	54.0	1.0	7.0	88.4926	98.4124

Figure 6 shows a histogram to show the number of shots taken from each distance, with the orange colored bars representing shots that resulted in goals. This shows that there is a right-skewed distribution in the histogram for all the shots, whether the result in a goal or not. This can indicate that players in the NHL prioritize getting to areas between 0 and 40 feet from the net to give them a better chance of scoring a goal.

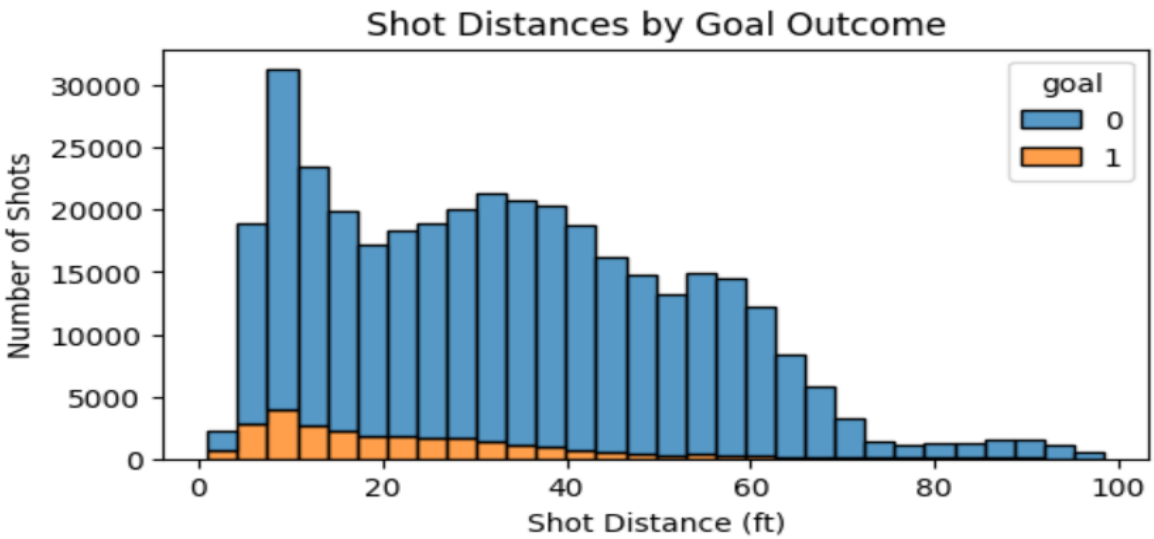

Figure 6: Shot Distance Histogram

Figure 7 shows the different types of shots taken on the x-axis, the number of shots taken on the y-axis, and a color coding of green represents shots that resulted in goals, and a color coding of red represents shots taken without a goal scored. The wrist shot appears to be the most effective shot type in the NHL, as it has over 175,000 shots taken, which represents almost half the amount of shots in the whole data set. A wrist shot is the simplest shot that requires the least amount of time to get off the stick compared to other shot types. Snap shot is also another shot that takes less time to shoot off the stick, which explains why it ranks second. The bars for other shot types decrease significantly

as you get to the more rare shot types like backhand and wrap-around. The NHL is the most skilled league in the world, and there is not enough time in a fast-paced game like hockey to produce more rare shots on net. The histogram may not be surprising, but it is a foundation to the project that will help understand every aspect of a shot produced in the NHL.

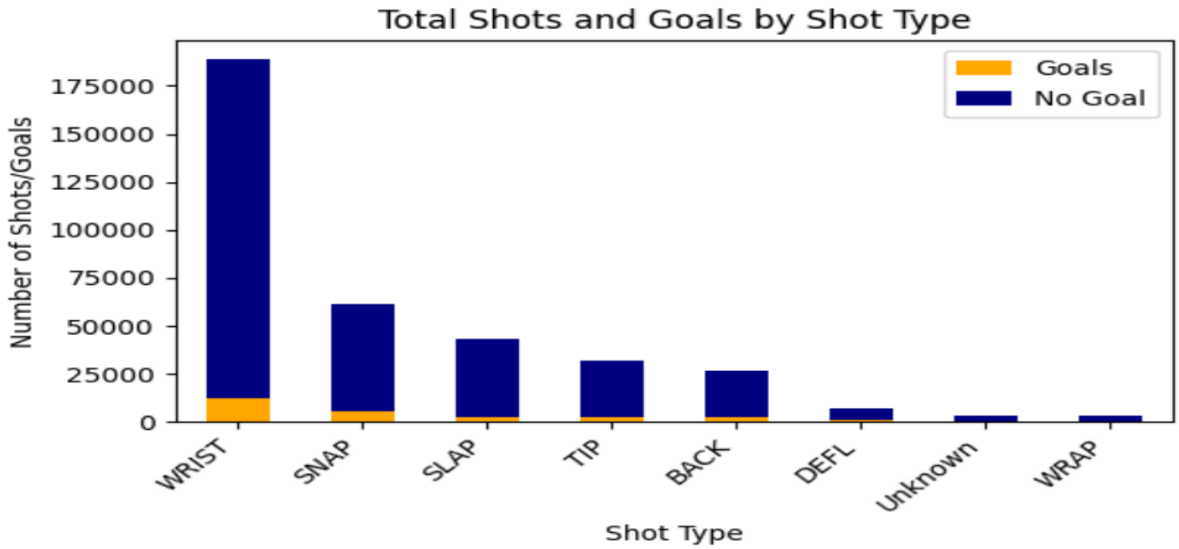

Figure 7: Shot Types Histogram

Since the feature 'goalieNameForShot' ranked as a top feature in my feature importance models, I wanted to include a table that shows the top ten goalies in the NHL, Table 4. These top ten goalies all have save percentages around 94%. I made the ranking so that there was a minimum of 500 shots faced, excluding any outliers. This table only ranks the goalies off on save percentage, meaning these are the goalies who have the best goals over shot percentage. It is interesting to see how close their save percentages are to each other. This gives an understanding that the NHL is a top league in the world, and goalies who don't have good save percentages typically don't last long in the NHL. This table gives the project a good example of how unlikely shots are to result in a goal, especially in the best league in the world.

Table 4: Top 10 Goalies by Save Percentage (minimum 500 shots faced)

Goalie	**Total Shots Faced**	**Goals Against**	**Saves**	**Save %**
Jet Greaves	1046	54	992	94.837476
Laurent Brossoit	1712	91	1621	94.684579
Anthony Stolarz	3495	192	3303	94.506438
Connor Hellebuyck	8788	504	8284	94.264907
Linus Ullmark	6096	352	5744	94.225722
Filip Gustavsson	6583	384	6199	94.166793
Joey Daccord	4755	280	4475	94.111462
Adin Hill	5818	343	5475	94.104503
Semyon Varlamov	2860	170	2690	94.055944
Ilya Sorokin	8089	485	7604	94.004203

Table 5 shows the top ten NHL goal scorers from years 2022-2025 with their shooting percentage (goals over shots) for comparison. All of these players have produced over 900 shots within 3 NHL seasons. The lowest shooting percentage of these top ten goal scorers is Nathan MacKinnon, who has 8.64% shooting percentage, but he has the second most shots produced. The best goal scorer in the NHL is Leon Draisaitl. Draisaitl may not have the best shooting percentage in the league, but he produced 1279 shots in three seasons with an outstanding number of 179 goals. Many factors go into a player in the NHL being a top goal scorer, but for the purpose of the analysis, we are looking at the shot production from these players.

Table 5: Top 10 NHL Players by Total Goals and Shooting Percentage

Shooter Name	**Total Shots**	**Total Goals**	**Shooting Percentage (%)**
Leon Draisaitl	1279	179	13.99
David Pastrnak	1622	158	9.74
Sam Reinhart	1307	156	11.94
Auston Matthews	1554	151	9.72
Mikko Rantanen	1319	149	11.30
Brayden Point	980	145	14.80
Connor McDavid	1363	143	10.49
Zach Hyman	1309	140	10.70
Nathan MacKinnon	1609	139	8.64
William Nylander	1316	138	10.49

Table 6 strictly goes off of shooting percentage in the NHL. To keep the analysis accurate, I coded the number of shots to be at least 50 in order to be included in the table. Players that are included in the top ten for goal scoring and top ten in shooting percentage include Brayden Point and Leon Draisaitl. Brayden Point had the best shooting percentage at 14.8, this is impressive, as he generated 980 shots. Marc Gatcomb had a 14.55 shooting percentage, ranking him second in the NHL, but he only took 55 shots within the three years I analyzed. This represents that Gatcomb has played in far fewer games than NHL players like Point and Draisaitl, so he has been up and down from the affiliate league to the NHL. Sam Colangelo and Aatu Raty also appear to be in similar situations, as they have both produced fewer than 100 shots as well. As said above there are multiple factors that go into shooting percentage. Just because somebody has a good shooting percentage it does not mean they are a top NHL goal scorer as we saw that only two of the top ten NHL goal scorers appear to be in the top ten shooting percentage. These results will help guide the project to look deeper into NHL shot analysis.

Table 6: Top 10 Shooters by Shooting Percentage (minimum 50 shots)

Shooter Name	**Total Shots**	**Total Goals**	**Shooting Percentage (%)**
Brayden Point	980	145	14.80
Marc Gatcomb	55	8	14.55
Sam Colangelo	76	11	14.47
Adam Gaudette	142	20	14.08
Leon Draisaitl	1279	179	13.99
Aatu Raty	66	9	13.64
Mark Scheifele	851	114	13.40
Sonny Milano	195	26	13.33
Gabriel Vilardi	564	75	13.30
Jonathan Toews	113	15	13.27

To build off the top goal scorers and shot percentage tables above, I decided to create a shot plot, Figure 8, in Python using only Leon Draisaitl's goals. Each dot on the plot represents a different goal Leon scored relative to his positioning on the ice. The coloring of the dots represents which type of shot he took when scoring the goal.

It appears that Leon scored a lot of snap-shot goals, as we can see a lot of orange. The location of most of his goals is mostly on the right side of the rink, which indicates he is shooting the puck from the right side as a lefty shooter. Since Leon is on the power play, that tells me that he is scoring a lot of goals, taking one-timers that are from either slap shots or snap shots, as the right side is mostly orange or red.

A talent like Leon Draisaitl is hard to come by; it certainly helps that Conner McDavid is on the same team, as he is very dynamic with the puck and makes plays that set Leon up with a cross-ice pass forcing the opposing goalie to move laterally in order to save a Draisaitl shot. Leon Draisaitl is a great example for my project because he understands when to shoot the puck and what location on the ice to shoot it from. Draisaitl is the most dominant shooter from seasons 2022-23 to season 2024-25 as he has produced 179 goals with a shooting percentage of 13.99.

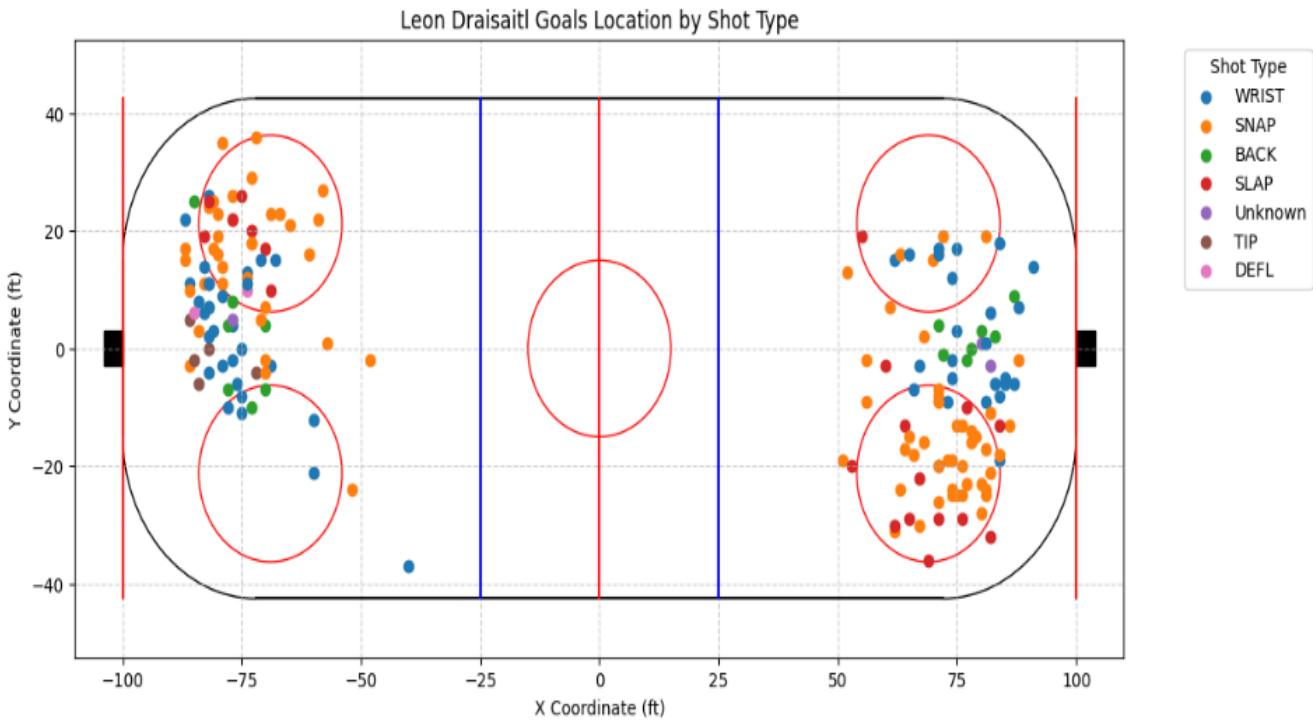

Figure 8: Leon Draisaitl Goal Plot

The exploratory analysis revealed patterns related to shot location, shot distance, shot type, and goalie skill level. NHL players generate most of their shots 40 feet and in, while relying mostly on wrist and snap shots. Goal scoring is rare, as only 7% of shots result in goals, emphasizing the difficulty of scoring and the importance for players to identify high-quality scoring areas. These findings help bring the feature selection and model design together for the rest of the analysis.

6 Statistical Analysis & Predictive Modeling

6.1 Formulas

For the predictive and testing part of the analysis, several tests were conducted. First, all numerical variables were evaluated using a one-way ANOVA F-test to find which variables are significant to goal and non-goal outcomes. The ANOVA F-statistic compares "between-group variance" to "within-group variance", and is computed as shown in equations (1)-(3):

$$F = \frac{\mathrm{MS}_{\text{between}}}{\mathrm{MS}_{\text{within}}} \tag{1}$$

where

$$\mathrm{MS}_{\text{between}} = \frac{\sum_{i=1}^{k} n_i \left(\bar{x}_i - \bar{x}\right)^2}{k-1} \tag{2}$$

and

$$\mathrm{MS}_{\text{within}} = \frac{\sum_{i=1}^{k}(n_i - 1)s_i^2}{N - k} \tag{3}$$

Similarly, I used all the categorical variables for a Chi-squared test. The Chi-squared statistic equation (4):

$$\chi^2 = \sum_{i=1}^{r}\sum_{j=1}^{c}\frac{(O_{ij} - E_{ij})^2}{E_{ij}} \tag{4}$$

where O_{ij} and E_{ij} represent the observed and expected frequencies in each cell of a $r \times c$ contingency table.

After the statistical tests, Logistic Regression was used to estimate the probability of a goal (1) or no-goal (0), equation (5):

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k \tag{5}$$

where $X_1, X_2, \ldots, X_k$ are the predictor variables, and $\beta_0, \beta_1, \ldots, \beta_k$ are the model coefficients. The estimated probability of a goal is provided in equation (6):

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k)}} \tag{6}$$

The next test performed was on the XGBoost model, which is a gradient-boosted decision tree framework that builds trees, correcting the errors of the previous ones, while regularizing the loss function to prevent overfitting. The function can be written as shown in equations (7) - (8):

$$\mathcal{L} = \sum_{i=1}^{N} l(y_i, \hat{y}_i) + \sum_{t=1}^{T}\Omega(f_t) \tag{7}$$

with the regularization term:

$$\Omega(f_t) = \gamma T + \frac{1}{2}\lambda\|w\|^2 \tag{8}$$

where $l(\cdot)$ is the loss function (logistic loss for classification), T is the number of trees, γ penalizes complexity, and λ controls L2 regularization on leaf weights.

The next function to make note of is the Decision Tree. The Decision Trees work by splitting feature space into regions, minimizing impurities. The function is shown in equations (9) - (10):

$$G = 1 - \sum_{k=1}^{K} p_k^2, \tag{9}$$

where p_k is the proportion of class k in the node. The optimal split is the feature and threshold that minimizes the weighted impurity:

$$\text{Split}^* = \arg\min_s \left(\frac{n_L}{n} G_L + \frac{n_R}{n} G_R \right), \tag{10}$$

where n_L and n_R are the left/right node sample sizes.

Random Forest is the last prediction method I will be using. Random Forest improves Decision Trees by averaging predictions from many bootstrapped trees. The function is shown in equation (11):

$$\hat{y} = \text{mode}\left(h_1(x), h_2(x), \ldots, h_B(x)\right), \tag{11}$$

where each $h_b(x)$ is a tree trained on a bootstrap sample and a random subset of features.

6.2 ANOVA F-Test

To interpret the results from Table 7, we have to understand that a feature is significant if the p-value is lower than 0.05. Based on the ANOVA results, 'arenaAdjustedShotDistance', 'shotDistance', period, and 'isHomeTeam' are all significant features to determining if a shot will result in a goal. Both 'arenaAdjustedShotDistance' and 'shotDistance' have extremely small p-values, indicating that there is a big difference in non-goal shot distance averages to goal shot differences. The feature 'period' also has a p-value that is way more than significant at 1.848e-58, meaning that the period the shot was taken in has a significant impact on a goal being scored or not. 'isHomeTeam' is the last significant predictor at a p-value of 0.0134, this suggests that home team or not does impact whether a shot results in a goal or not. Features 'arenaAdjustedYCord', 'shotAngle', and 'arenaAdjustedXCord' are all not significant because their p-value scores are above the significance level at 0.05.

Table 7: ANOVA F-test Results

Feature	**F-statistic**	**p-value**
arenaAdjustedShotDistance	9366.383	0.000000×10^0
shotDistance	9397.353	0.000000×10^0
period	259.945	1.848224×10^{-58}
isHomeTeam	6.115	1.340455×10^{-2}
arenaAdjustedYCord	1.763	1.842494×10^{-1}
shotAngle	1.052	3.051191×10^{-1}
arenaAdjustedXCord	0.094	7.588629×10^{-1}

6.3 Chi-Squared Test

Similar to the ANOVA test above, for our categorical variables, we look at the p-value to interpret the Chi-squared test results in Table 8. If the p-value is below 0.05, we

can reject the null hypothesis and determine that the categorical feature is relevant to telling a difference between goals and no goals. Looking at the table, we found that all of the features are below a p-value of 0.05 except for season. Variables 'goalieNameForShot', 'playerPositionThatDidEvent', 'shooterName', and 'strengthSituation' all have an extremely small p-value, which indicates that there is a high statistical significance between these categorical features and the occurrence of a goal. In other words, the goalie, player position, shooter's name, and strength of the situation (power play, penalty kill, or even strength) are all strongly related to a shot becoming a goal or not. Variables 'lastEventCategory', 'shotType', 'location', 'homeTeamCode', and 'team' are also very significant predictors for goals being scored, as they are all below the 0.05 p-value. The last event, type of shot, location, home team, and team all closely relate to a shot resulting in a goal. The variable 'season' has a p-value of 0.1207 so we can say that the season the shot takes place in is not important to predicting a goal.

Table 8: Chi-squared Test Results

Feature	**Chi^2 Statistic**	**p-value**
goalieNameForShot	10172.628	0.000000×10^{0}
shooterName	4700.376	0.000000×10^{0}
playerPositionThatDidEvent	2689.327	0.000000×10^{0}
strengthSituation	2138.590	0.000000×10^{0}
lastEventCategory	1293.116	$1.365709 \times 10^{-270}$
shotType	1174.088	$2.831413 \times 10^{-249}$
location	262.695	9.044916×10^{-58}
homeTeamCode	65.688	4.114949×10^{-4}
team	6.083	1.364894×10^{-2}
season	4.230	1.206624×10^{-1}

The statistical tests show that there are several relationships that are significant relationships between goal outcomes from our numerical and categorical variables. ANOVA identified shot distance and period of the game to be the most influential numerical predictors. Chi-squared results show that shooter identity, goalie identity, and game situation as the strongest categorical predictors. These tests confirm that features do differentiate goals from non-goals and show why they are also important in model classification.

6.4 Logistic Regression Analysis

The Logistic Regression model had an overall accuracy of 0.605, indicating that the model correctly classified 60% of shots as goals or non-goals, shown in Table 9. The model was good at predicting non-goal events, as the precision was 0.962, but the model struggled to predict shots resulting in a goal with a low precision of 0.116. The recall for goals was higher at 0.693, showing that the model captured a lot of goals but also produced a large number of false positive results. The true negatives meant there were 50,678 shots that were not goals, and the model correctly predicted them as non-goals. False positives resulted meant there being 33,968 shots that were not goals, but the model incorrectly predicted them as goals. A false negative result meant that there were 1,976 shots that were goals, but the model incorrectly predicted them as non-goals. Finally, there were 4,470 true positive shots that were actually goals, and the model correctly predicted them as goals. Refer to Figure 9 for the confusion matrix. Overall, logistic regression had a hard time predicting goals, but it is important for the analysis, so we can determine which models work better with my data set.

Table 9: Logistic Regression Classification Report for Goal Prediction

Class	**Precision**	**Recall**	**F1-score**	**Support**
0	0.962	0.599	0.738	84,646
1	0.116	0.693	0.199	6,446
Accuracy		0.605		
Macro Avg	0.539	0.646	0.469	91,092
Weighted Avg	0.903	0.605	0.700	91,092

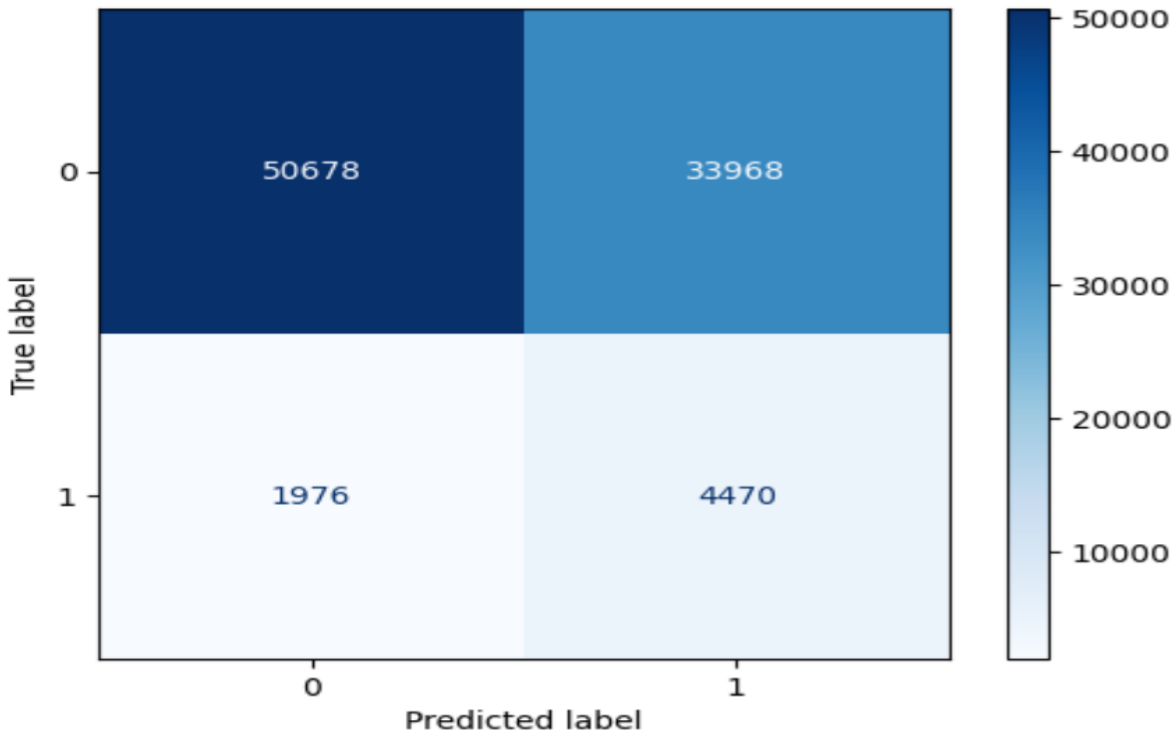

Figure 9: Logistic Regression Confusion Matrix

6.5 Decision Tree Analysis

The Decision Tree model scored an overall accuracy of 0.87, indicating the model correctly classified 87% of shots as goals or non-goals, Table 10. Similar to the other models, the Decision Tree did well at classifying non-goals with a precision of 0.94 and recall of 0.92. However, the model struggled to predict shots as a goal with a precision of only 0.13 and a recall of 0.16. The confusion matrix correctly predicted 77,832 shots as true negatives, meaning the non-goals were correctly identified. There were 6,814 false positives, meaning the model predicted goals when they were actually non-goals. The model had 5,385 false positives, meaning there were actual goals, but the model predicted them as non-goals. Last, the model correctly predicted 1,061 shots as goals. See Figure 10 for the visual of confusion matrix.

Table 10: Decision Tree Classification Report for Goal Prediction

Class	**Precision**	**Recall**	**F1-score**	**Support**
0	0.94	0.92	0.93	84,646
1	0.13	0.16	0.15	6,446
Accuracy		0.87		
Macro Avg	0.54	0.54	0.54	91,092
Weighted Avg	0.88	0.87	0.87	91,092

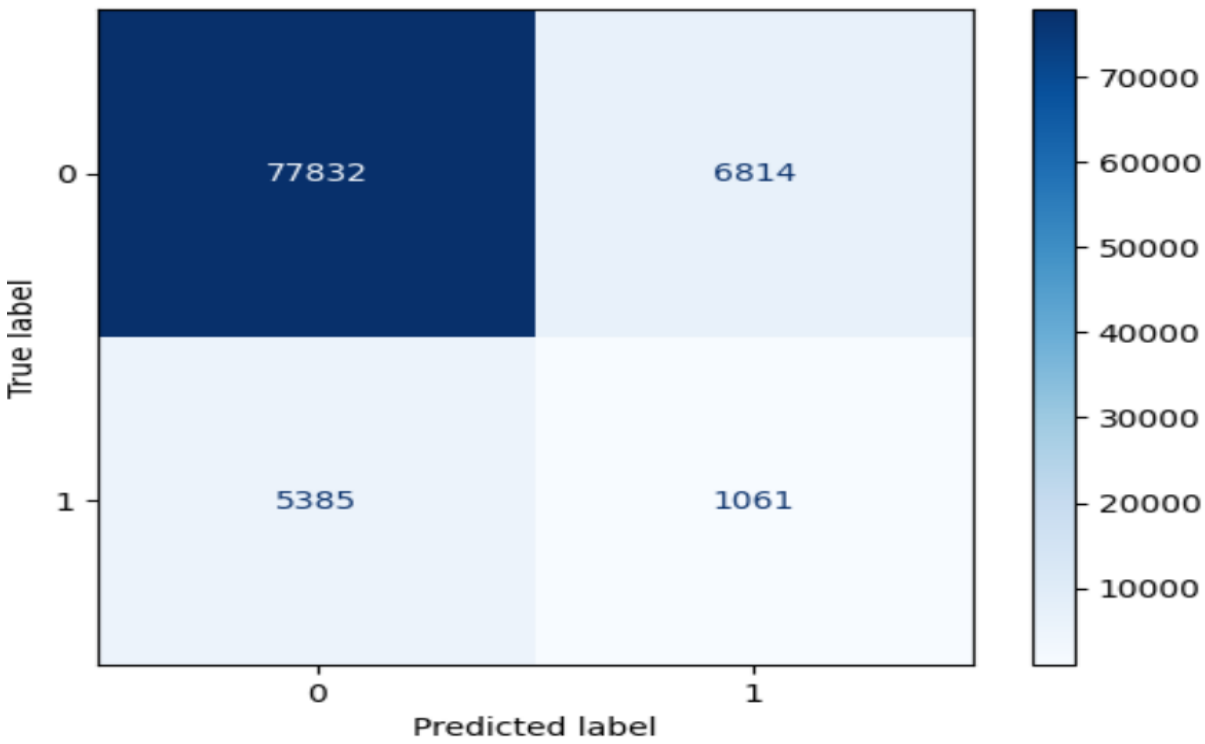

Figure 10: Decision Tree Confusion Matrix

6.6 Random Forest Analysis

Random Forest performed very well overall, scoring an accuracy of 0.93, representing the model correctly identified goals scored and non-goals, Table 11. The model scored 0.93 for non-goals with a recall of 1.00, meaning the model correctly identified just about every non-goal. However, the model struggled at predicting goals with a precision of 0.67 and a recall of 0.04. This meant that the model was correct 67% of the time and the model correctly identified 4% of actual goals. The confusion matrix highlights the imbalance better, see Figure 11. The model correctly predicted 84,508 non-goal shots as true negatives. Incorrectly predicted 138 shots as a goal for false positives. Missed 6,161 goals and labeled them non-goals for false negatives. Last, only 285 true positives, in other words, correctly predicting a goal.

Table 11: Random Forest Classification Report for Goal Prediction

Class	**Precision**	**Recall**	**F1-score**	**Support**
0	0.93	1.00	0.96	84,646
1	0.67	0.04	0.08	6,446
Accuracy		0.93		
Macro Avg	0.80	0.52	0.52	91,092
Weighted Avg	0.91	0.93	0.90	91,092

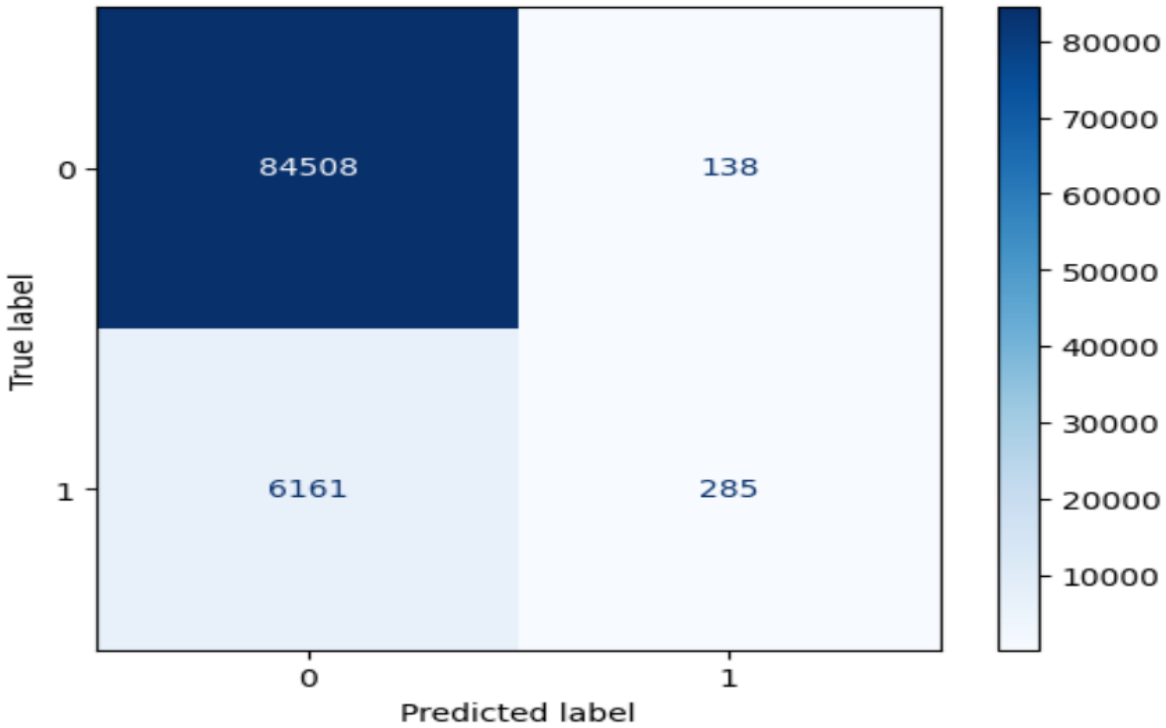

Figure 11: Random Forest Confusion Matrix

6.7 XGBoost Analysis

XGBoost is an advanced Machine Learning algorithm that builds its model based on a combination of many Decision Trees, and each new tree corrects the errors made by the previous ones. In context for the project, XGBoost learns patterns from thousands of shot attempts like shot distance, shot angle, period, location, player position, and more to estimate the probability that a given shot becomes a goal. The algorithm works by optimizing a loss function at the same time, regularizing to prevent overfitting. This makes the model powerful and stable for a complex play-by-play data set I chose. Overall, XGBoost can capture non-linear relationships and interactions between features way better than simple linear models can. This allows for modeling to be done on different game situations to influence goal probability. XGBoost is specifically effective for xG modeling, where many factors determine how good a chance a shot has in becoming a goal.

The XGBoost model's overall accuracy is rounded up to 93%, which is way better than the logistic regression model at 60%. This model includes both goals scored and no goals scored. The precision for no goal was 93%, while the precision for a goal was only 65%, meaning that the XGBoost model was correct 65% of the time for predicting a goal scored, Table 12. Looking at the confusion matrix in Figure 12, the model correctly predicted 84,471 no goals. The confusion matrix had 175 false positives, which meant that the model predicted a goal when there wasn't. There were 6,128 false negatives, meaning the model predicted no goal when there was a goal. Last, there were 318 true positives, meaning the model correctly predicted a goal. Overall, both models were very biased towards predicting non-goals, which is expected because of how imbalanced the goal-to-non-goal ratio is. The performance shows how challenging it can be to predict the rare event of a goal being scored in a hockey shot data set. This stresses the importance of evaluating more than the overall accuracy of a model when analyzing data.

Table 12: XGBoost Classification Report

Class	**Precision**	**Recall**	**F1-score**	**Support**
0	0.93	1.00	0.96	67638
1	0.65	0.05	0.09	5236
Accuracy		0.93		
Macro Avg	0.79	0.52	0.53	72874
Weighted Avg	0.91	0.93	0.90	72874

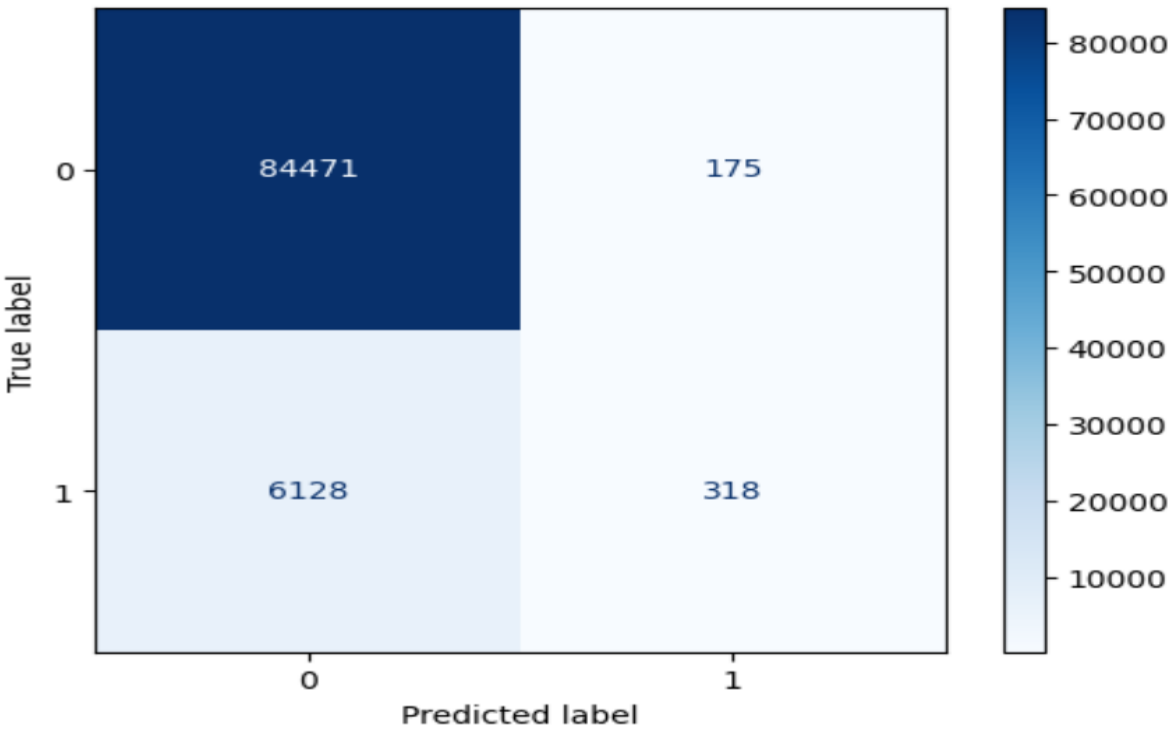

Figure 12: XGBoost Confusion Matrix

6.8 ROC-AUC

To further evaluate the model's ability to distinguish goals and non-goals, I calculated the Receiver Operating Characteristic- Area Under the Curve(ROC-AUC) and Precision-Recall AUC scores. These metrics are important because of the imbalanced dataset I have, where goals only represent 7% of all events. Different from accuracy, AUC metrics give a more reliable measure of how well the model separates the positive class from the negative class across all possibilities.

The Logistic Regression ROC-AUC, Figure 13, score of 0.69 indicates that the model correctly ranks randomly chosen goals above a randomly chosen non-goal 69% of the time. The precision-recall AUC score is 0.1298. This indicates that it is difficult to predict rare events like goals. Overall, these scores show that logistic regression does capture a portion of the features but has limited predictive power for goals due to the class imbalance.

```
ROC-AUC Score: 0.6903
Precision-Recall AUC Score: 0.1298
```

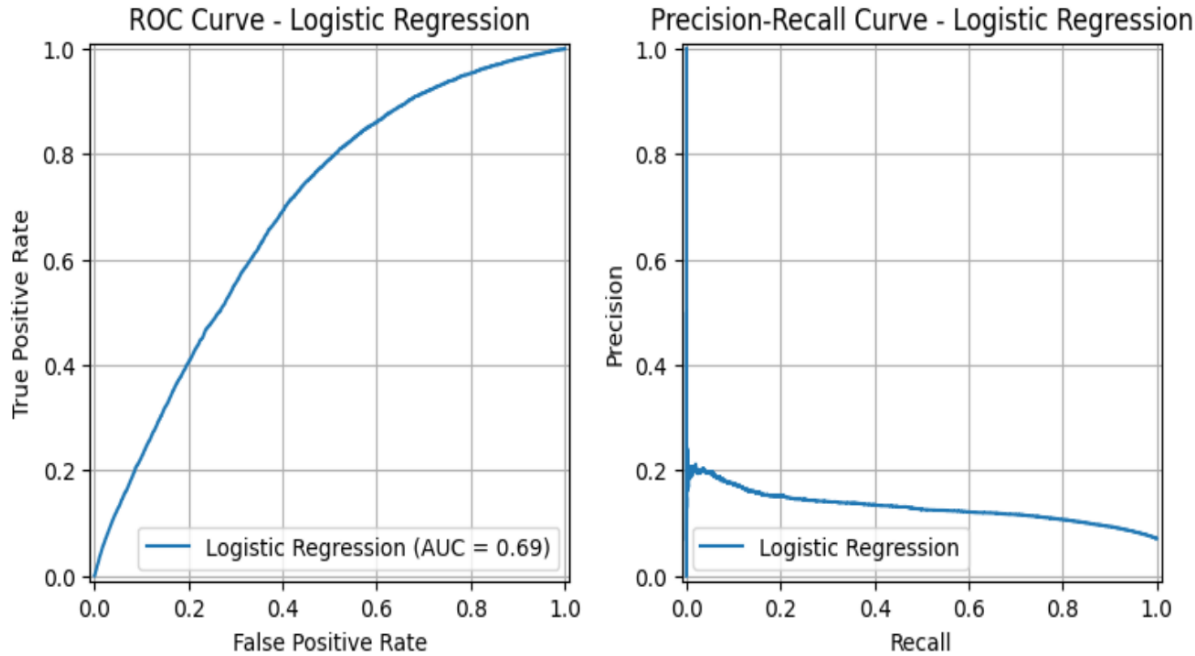

Figure 13: Logistic Regression ROC-AUC Curve & Precision Recall Curve

The Decision Tree ROC-AUC, Figure 14, score of 0.5420 indicates that the Decision Tree struggled to distinguish between goals and non-goals. The Precision-Recall AUC score of 0.1792 was even lower, showing how difficult it was to predict goals.

```
Decision Tree ROC-AUC Score: 0.5420
Decision Tree Precision-Recall AUC Score: 0.1792
```

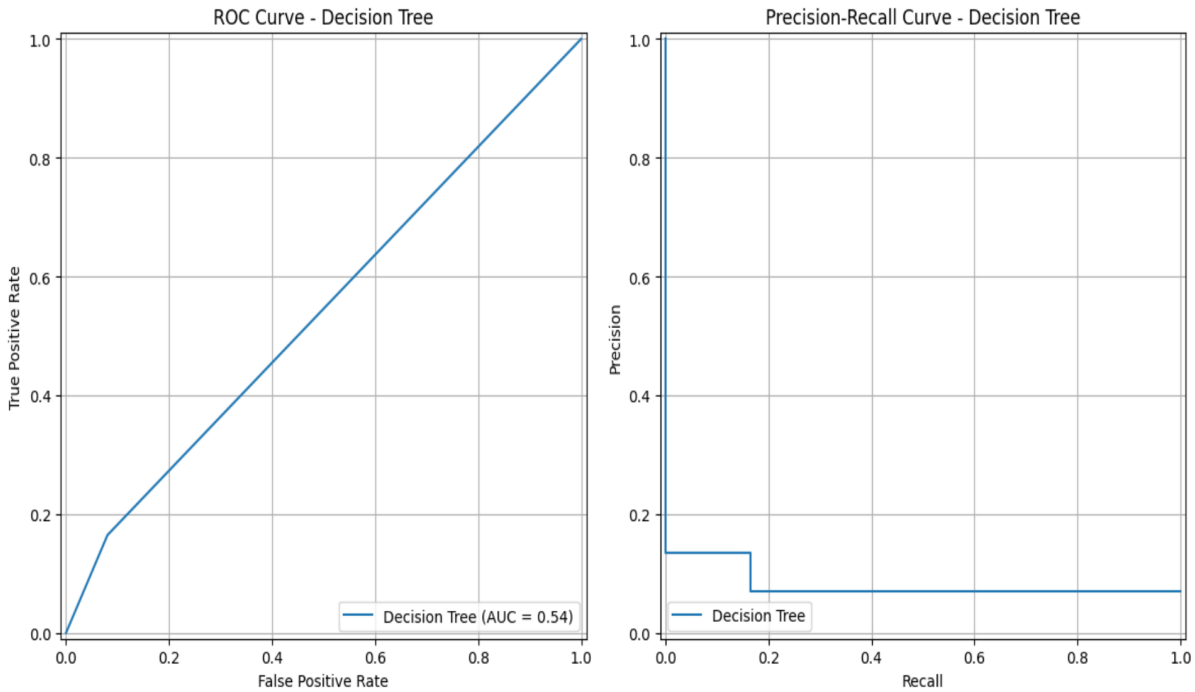

Figure 14: Decision Tree ROC-AUC Curve & Precision Recall Curve

The Random Forest model in Figure 15 had stronger performance than the Decision Tree did with a ROC-AUC score of 0.7292. This value shows that the model is distinguishing goals and non-goals better than random chance and shows how averaging in Random Forest improves the results. Although, Precision-Recall AUC score of 0.2057 is not great and shows it still struggles because of the imbalanced dataset. There is improvement using Random Forest, but like the other models, it struggled detecting goals(1).

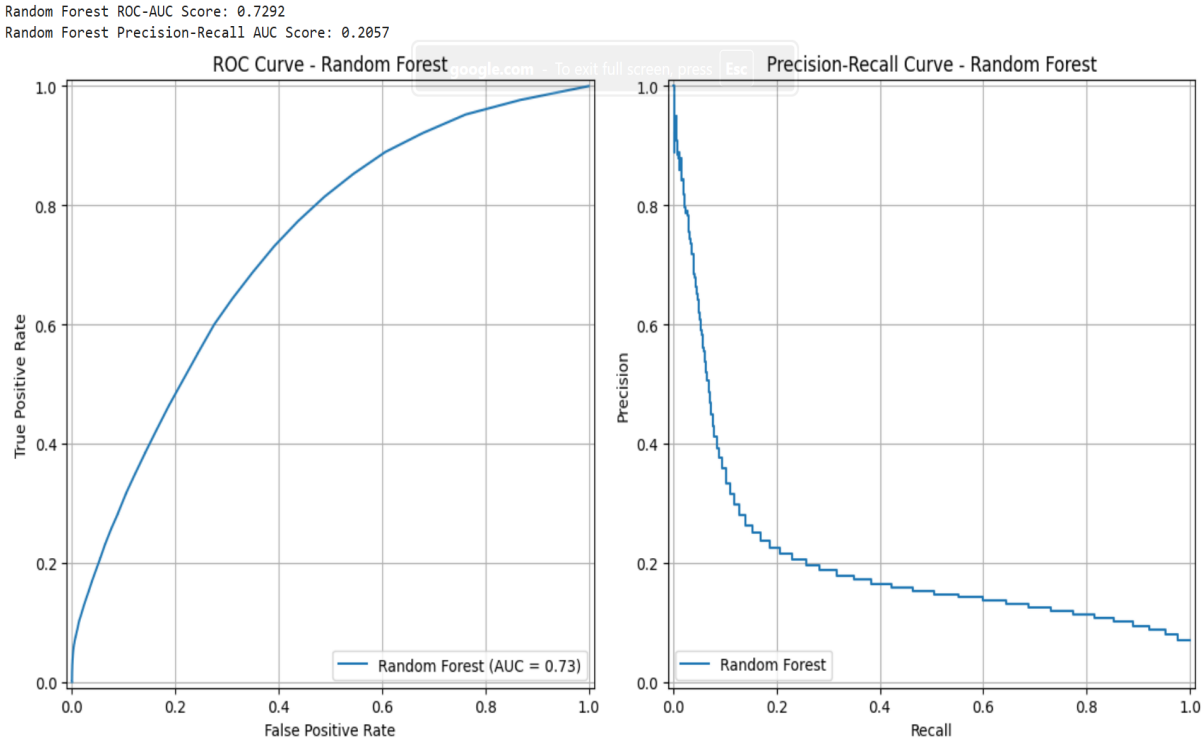

Figure 15: Decision Tree ROC-AUC Curve & Precision Recall Curve

The XGBoost model had an ROC-AUC, Figure 16, score of 0.8107, indicating the model correctly ranks a randomly chosen goal above a randomly chosen non-goal 81% of the time. The precision-recall AUC score of 0.313 shows the XGBoost model's ability to identify rare events like goals. Even though the score is still moderate, it is a significant improvement compared to the logistic regression score (0.1298).

```
XGBoost ROC-AUC Score: 0.8107
XGBoost Precision-Recall AUC Score: 0.3130
```

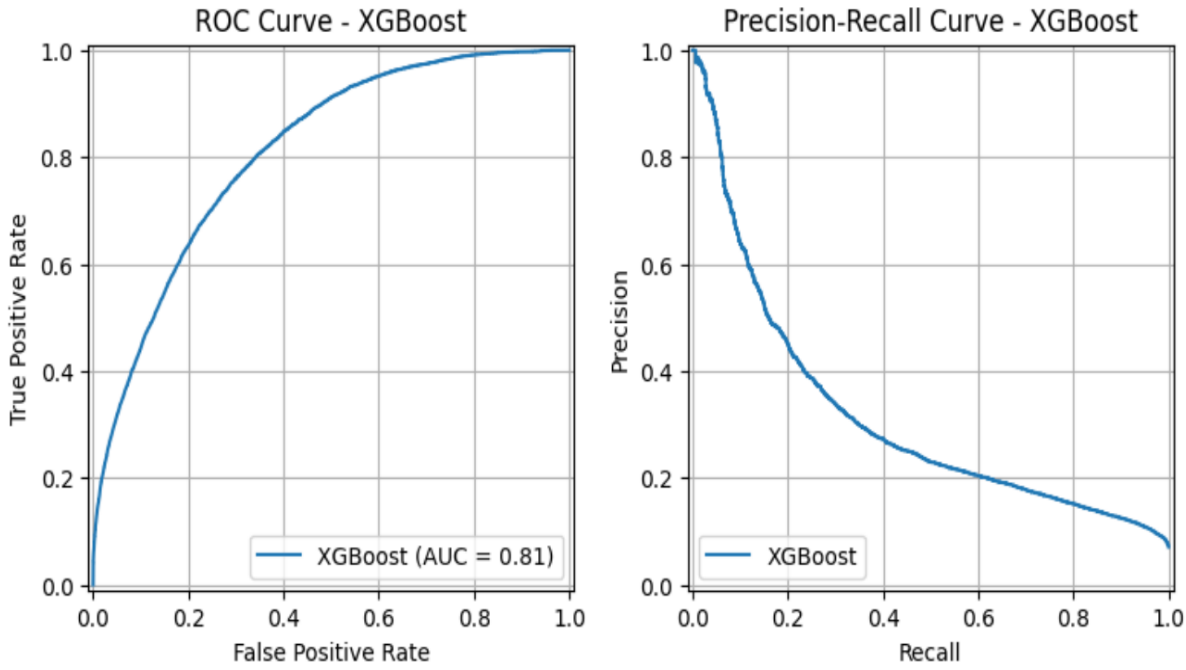

Figure 16: XGBoost ROC-AUC Curve & Precision Recall Curve

Overall, these metrics show that the XGBoost model turns out to be more effective compared to our other models for predicting NHL goals, but it is still a challenge. Scoring in hockey is influenced by more factors than the data set used has, such as traffic in front of the net, passing sequences, defense, and decision-making. This reinforces the idea that analytics can provide good insights into shot quality and scoring probability, but it is almost impossible to predict scoring. Ultimately, the models highlight patterns, not certainty, showing how a sport like hockey has too much complexity for predicting rare events.

7 Discussion

7.1 Classification Results

After using the three classification models (Logistic Regression, Random Forest, and Decision Tree) I found some patterns that influence the likelihood of a shot becoming a goal. Each model works a bit differently from the other, but the ovearll findings give a clear picture to which features matter most in terms of scoring a goal.

A common result that I found is a strong influence is the individual player skill. Random Forest and Decision Tree ranked 'shooterName' and 'goalieNameForShot' as the two most important predictors. This shows that the offensive and defensive abilities of NHL players have a major impact on the odds of a goal happening.

Spatial variables like 'shotDistance' and 'shotAngle' also appeared to be important pre-

dictors. The Logistic Regression model ranked 'shotDistance' as the best predictor; both Random Forest and Decision Tree ranked 'shotDistance' third. 'shotAngle' ranked 4th in Random Forest and Decision Tree. Together this shows that the distance from the net and the angle a shot has from the net play a significant role in boosting the odds of a goal.

Context variables like 'location', 'team', and 'lastEventCategory' also appeared to be top predictors. This shows that the better the team is, the more likely they are to score a goal. An event like a turnover or rebound is important for resulting in a goal, as well as the location of where the puck is shot on the ice. These variables were not ranked as high as the other four, but they are important to recognize for our analysis.

In general, the three models used indicate that the scoring of goals is the result of a combination of individual skill level, shot quality, and contextual game factors. The skill of the player and the location of the shot resulted as the most dominant predictors. Being able to see consistent results from both Random Forest and Decision Tree strengthens the results that were found. Logistic Regression seemed to have given traditional hockey analytics results. Together, these findings give a good understanding of which variables strongly influence a shot to produce a goal.

The complete code for this project is provided in the Appendices (A-F)

7.2 Exploratory Analysis Results

Using results from the classification and predictive models, I decided to add exploratory analysis to the project to better understand what separates goals from non-goals in the NHL. The results are taken from descriptive statistics, shot distance metrics, shot type effectiveness, goalie performance, and individual shooting patterns. Together, the visuals and tables give context for interpreting predictive models and insight into NHL scoring.

The descriptive statistics show that out of 364,368 total shots in three years, only 7% resulted in goals. This shows how difficult it is to score in the NHL. The average shot distance was around 34 feet, suggesting that most shots are taken between the blue line and the hash marks. Further, the coordinates and shot-angle metrics centered around zero indicate a balance of shooting patterns from both sides and angles of the ice. This supports that teams prioritize generating shots from closer to mid-range from the net.

The shot distance histogram shows there is a right-skewed distribution, with more shot attempts coming from between 0 and 40 feet from the net. This indicates that players don't typically shoot the puck from far out, as they understand the odds that a goalie will save the puck are high. Together, the findings show that the quality of the shot is more important than the shot volume by itself.

The type of shot is also important to understanding shot analytics. Wrist shots make up close to half of all the recorded shots and are the most effective in terms of producing goals. The wrist shot is easier to release faster, and it is more accurate than other shot types. Similarly, snapshots rank second in effectiveness and in volume. Less common shots like backhands, wrap-arounds, and tip-ins occur less often just because of the situation. This

visual from Figure 7 demonstrates that hockey is a fast-paced sport and players rely on simple and effective shooting mechanics to score the majority of their goals.

Goaltender performance from Table 4 gives justification for how individual skill is important in determining a successful shot. The top ten NHL goalies in this dataset all have save percentages around 94%, demonstrating how good goalies are, especially in the NHL. Including this table was important because 'goalieNameForShot' ranked as one of the most important features in our three classification models. The save percentage explains how hard it is to get a puck past an NHL goalie, even from a strong scoring area.

To build on individual skill performance, shot performances from tables 5 and 6 give examples of the best goal scorers and the best shooters in the NHL. Players like Leon Draisaitl, David Pastrňák, Sam Reinhart, and Auston Matthews all generate high goal totals because of the volume and quality of their shot attempts. There is a difference between top goal scorers and top shooting percentage rankings. Just because a player is efficient at shooting the puck doesn't mean they are a top goal scorer. We see that the top goal scorers have decent shooting percentages, but their volume of shots taken is way higher than players who have high shooting percentages. All of course, with the exception of Brayden Point and Leon Draisaitl, as they ranked in the top ten for both.

To build off the shot tables, Figure 8 analyzes the goals scored by Leon Draisaitl. The plot reveals that Draisaitl takes most of his shots on his offhanded side, indicating he frequently takes one-timers from a high percentage scoring area. The shot plot visualization shows fits the other characteristics above; most of his goals are within the 40-foot range from the net, they are mostly snapshot or slap-shot, and the angle of the shot is consistently on the right side of the rink.

As a whole, the exploratory analysis highlights the player talent, shot selection, shot distance, angle of the shot, and goalie talent. These insights validate the feature importance rankings observed in the classification models, and they give us a better understanding of NHL shot behavior. All of this builds up to a stronger analytic approach to the end of the project, predictive modeling.

7.3 Statistical Analysis & Predictive Modeling

The ANOVA test results showed that numerical features like 'shotDistance' and 'arenaAdjustedDistance' are strong predictors of goal likelihood. As said above, the closer the shot is, the better chance it has at becoming a goal. The variable 'period' being a strong predictor also suggests that scoring changes throughout the times of the game. The last significant variable 'isHomeTeam' indicates that there is a slight home-ice advantage.

The chi-squared results show the importance of the categorical variables. 'goalieNameForShot', 'shooterName', 'playerPositionThatDidEvent', and 'strengthSituation' all had very large Chi-Square statistics, giving confirmation that individual skill and game situations have a large influence on whether a shot becomes a goal. The variable 'goalieNameForShot', being the most significant categorical variable, makes sense given how skilled the goalies are at the NHL level.

The logistic regression model turned out to have an accuracy of 0.605, showing a struggle with the imbalance in the dataset. To explain this, the precision of non-goals was 0.962, and goals were 0.116. The confusion matrix showed that there was a small number of true goals that were correctly predicted, and many non-goals were classified as goals. Logistic Regression provides interpretable results, but its ability to capture shot complexity is limited.

The Decision Tree model scored an accuracy of 0.87, which was better than the linear model, Logistic Regression. The precision for non-goals was high at 0.94, while goals were low at 0.13. The confusion matrix showed that a majority of non-goals were correctly predicted, but there were few goals correctly predicted, and many goals were missed. The ROC-AUC score of 0.5420 and the Precision-Recall AUC of 0.1792 highlight the difficulty in capturing goals. Decision Tree captured more complexity than Logistic Regression, but it was still limited in detecting the rare event.

The Random Forest model scored an overall accuracy of 0.93, showing improvement from both Logistic Regression and Decision Tree. The precision for non-goals was 0.93, and 0.67 for goals. The confusion matrix indicated that most non-goals were correctly predicted, but the model struggled to identify actual goals with a large false negatives. The ROC-AUC score of 0.7292 shows that the model did better at distinguishing between goals and non-goals. However, the Precision-Recall AUC scored a 0.2057, indicating it still struggled. Random Forest takes advantage of multiple trees to capture more complex patterns in the data, which improves overall performance, but still not as accurate with goal prediction.

The XGBoost model performed a lot better than the other Logistic Regression and Decision Tree did, as the accuracy score was 0.93. The precision for non-goals was 0.93, and goals were 0.64. The improvements show that XGBoost is able to handle more nonlinear patterns and interactions between several variables. Even though the results turned out better, the confusion matrix also had a lot of false negatives and missed a lot of true positives. This shows that the XGBoost model was very careful at predicting a goal because they are so rare, so it underestimated goals as they are occurring only 7% of the time.

Looking at the AUC results, XGBoost dominated with an ROC-AUC score of 0.8107 and a PR-AUC of 0.313. I thought the Random Forest model would perform the best until I saw that the Precision-Recall AUC score was 0.1 lower than the XGBoost model. The precision-recall curve for XGBoost actually doubled compared to the Logistic Regression and Decision Tree, indicating it handles goals more effectively. This shows that it is important to account for all aspects of a model, not just the precision and accuracy from the classification reports.

7.4 Reflections

The results of this analysis show how complex it is to predict a goal in the NHL. I was not surprised when I got the results that I did, as I knew it would be very challenging to predict. Hockey is a fast-paced game that is so unpredictable for many different reasons. The fact that individual skill levels turned out to be the model's top predictor shows you

that goals are random, and it depends on how the individuals in the game are performing.

The analysis makes a valid point that scoring events are rarely a result of one single variable alone. Scoring is the product of decision-making, player shooting ability, goalie ability, and offensive zone systems. The XGBoost and Random Forest models were very solid at predicting features that are most important, but they still struggled with identifying goals. There are so many other variables and elements that could expand this project that are not in the datasets I used.

Another takeaway is how the imbalanced goals to non-goals were. Since only 7% resulted as a goal, the models naturally lean towards predicting no goal. Even the elite players score only a small percentage of their shot attempts, and the best goalies allow goals in bizarre moments.

Overall, this analysis reinforces the idea that data can reveal good patterns, but it cannot fully capture the complexity and creativity of a sport. Hockey analytics continues to grow, but predicting events such as goals is one of the hardest tasks; this project shows why.

Conclusion

The project analyzed over 364,000 NHL shot events from 2022-2025 seasons to find what influences goal scoring and evaluate which statistical models work better at predicting shot outcomes. The descriptive analysis gives context on shot distances, angles, periods, and shot locations. Feature importance ranks multiple classifiers, showing how both spatial and player-specific variables are critical in determining goal probability.

Using predictive modeling and statistical methods for testing, this study demonstrated that basic models like Logistic Regression can capture general linear trends, while more advanced models like XGBoost and Random Forest provide stronger predictive ability because of their ability to handle complexity. However, all four models struggled to correctly identify the rare event of a goal because of how imbalanced the class is. These limitations show the difficulty of predicting NHL scoring, where fast-paced, complex play and elite defending take place.

Overall, the combination of testing, classification models, and exploratory analysis resulted in an in-depth exploration of NHL shot data. The findings emphasize how distance, angle, player shooting, goalie facing shot, and game situation are all important for predicting shot likelihood. This project provides a strong foundation for analyzing other measures in a hockey game that were not included in the datasets used. In conclusion, predicting goals is very challenging, but the analysis gives insight into how scoring opportunities develop in the best hockey league in the world.

Acknowledgment

I would like to acknowledge the people who supported me throughout the process. First, I would like to thank my family and girlfriend for their constant support academically, financially, and emotionally.

I would like to express my gratitude to Dr. Yasser Alginahi of Adrian College for his consistent guidance, encouragement, and professionalism. His instruction each week helped me stay on track, stay focused, improve the clarity of my work, and learn how to communicate my findings effectively to any audience.

Finally, I would like to thank my fellow colleagues, Jonah Watson and Seth Logan were involved in helping my project every week, giving suggestions, sharing ideas, and allowing me to learn from their own capstone work. Their feedback played a large role in shaping this project.

References

[1] "Moneypuck data download." [Online]. Available: https://moneypuck.com/data.htm

[2] "Chatgpt." [Online]. Available: https://chatgpt.com/

[3] "Google gemini." [Online]. Available: https://gemini.google.com/

[4] R. C. McLean and M. R. Veall, "Performance and salary differentials in the national hockey league," *Canadian Public Policy*, vol. 18, no. 4, pp. 470–470, Dec. 1992.

[5] L. Donaldson, B. Li, and M. D. Cusimano, "Economic burden of time lost due to injury in nhl hockey players," *Injury Prevention*, vol. 20, no. 5, pp. 347–349, Jan. 2014.

[6] P. Lambrix, N. Carlsson, and R. Säfvenberg, "Goal-based performance metrics for ice hockey accounting for goal importance," *Journal of Sports Analytics*, vol. 11, Jan. 2025.

[7] S. N. Riley, "Investigating the multivariate nature of nhl player performance with structural equation modeling," *PLoS ONE*, vol. 12, no. 9, p. e0184346, Sep. 2017.

[8] N. Pelloneová, "Measuring the technical efficiency of hockey legends: Who is the best nhl player of all time?" *Studia Sportiva*, vol. 18, no. 1, Apr. 2024.

[9] B. C. Luu, A. L. Wright, H. S. Haeberle, J. M. Karnuta, M. S. Schickendantz, E. C. Makhni, B. U. Nwachukwu, R. J. Williams, and P. N. Ramkumar, "Machine learning outperforms logistic regression analysis to predict next-season nhl player injury," *Orthopaedic Journal of Sports Medicine*, vol. 8, no. 9, Sep. 2020.

[10] R. Moreau, H. Perera, and T. B. Swartz, "Valuation of nhl draft picks using functional data analysis," *Journal of Sports Analytics*, vol. 11, Mar. 2025.

[11] J. Wu, Y. Shen, D. Qian, and Y. Cui, "Effect of performance determinants on nhl final goal difference," *Revista Brasileira de Medicina do Esporte*, vol. 30, no. 2, Dec. 2023.

[12] G. Pischedda, "Predicting nhl match outcomes with ml models," *International Journal of Computer Applications*, vol. 101, no. 9, pp. 15–22, Sep. 2014.

[13] R. Chachad, S. Pradhan, and A. P. Medina, "This is fine: The impact of blowouts on subsequent game performance in the nhl," *Frontiers in Sports and Active Living*, vol. 5, Jan. 2024.

[14] O. Siivonen, "Predicting player performance in the national hockey league using data-analytics and advanced metrics," 2024.

[15] J. Viljakainen, "Quantifying success in the nhl: Statistical measures and predictive modeling," 2025.

[16] M. Pitassi and R. Cohen, "Advancing nhl analytics through explainable ai," in *Canadian AI*, 2024.

[17] M. A. DiBenedetto, "Skating for a payday: Analyzing nhl player performance in contract years," *Bryant University*, 2025.

[18] G. Liu, W. Zhu, and O. Schulte, "Interpreting deep sports analytics: Valuing actions and players in the nhl," in *International Workshop on Machine Learning and Data Mining for Sports Analytics*. Springer, 2018, pp. 69–81.

[19] U. Johansson, E. Wilderoth, and A. Sattari, "How analytics is changing ice hockey," in *Linköping Hockey Analytics Conference*, 2022, pp. 49–59.

[20] C. M. Farlinger, L. D. Kruisselbrink, and J. R. Fowles, "Relationships to skating performance in competitive hockey players," *The Journal of Strength & Conditioning Research*, vol. 21, no. 3, pp. 915–922, 2007.

[21] M. A. Gaudiani, J. P. Castle, M. J. Abbas, M. J. Myhand, T. J. Sprys-Tellner, J. T. McConnell, B. A. Pratt, and T. S. Lynch, "High return to play and variable salary impact after hip arthroscopy in national hockey league players," *Arthroscopy, Sports Medicine, and Rehabilitation*, vol. 7, no. 1, p. 101006, 2025.

[22] S. E. Buttrey, A. R. Washburn, and W. L. Price, "Estimating nhl scoring rates," *Naval Postgraduate School*, 2011.

[23] B. Macdonald, "An expected goals model for evaluating nhl teams and players," in *Proceedings of the 2012 MIT Sloan Sports Analytics Conference*, 2012.

[24] T. Müller-Schoell and L. Moustakas, "National diversity and nhl team performance, 2007–2011," *Choregia*, vol. 12, no. 1, pp. 15–27, 2016.

[25] T. C. Chan, J. A. Cho, and D. C. Novati, "Quantifying the contribution of nhl player types to team performance," *Interfaces*, vol. 42, no. 2, pp. 131–145, 2012.

[26] "Concept — classification algorithms." [Online]. Available: https://knowledge.dataiku.com/latest/ml-analytics/ml-concepts/concept-classification.html

Appendices

Appendix A: Data Cleaning

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, brier_score_loss
from sklearn.preprocessing import OneHotEncoder
import xgboost as xgb

# Load datasets
shots_22 = pd.read_csv("/content/drive/MyDrive/Capstone/shots_2022.csv")
shots_23 = pd.read_csv("/content/drive/MyDrive/Capstone/shots_2023.csv")
shots_24 = pd.read_csv("/content/drive/MyDrive/Capstone/shots_2024.csv")

# Add season labels
shots_22['season'] = '2022-23'
shots_23['season'] = '2023-24'
shots_24['season'] = '2024-25'

# Combine datasets
shots = pd.concat([shots_22, shots_23, shots_24], ignore_index=True)

# Variables to keep
keep_variables = [
    'goal', 'arenaAdjustedShotDistance', 'arenaAdjustedXCord',
    'arenaAdjustedYCord', 'isHomeTeam', 'goalieNameForShot',
    'homeTeamCode', 'lastEventCategory', 'location', 'period',
    'season', 'homeSkatersOnIce', 'awaySkatersOnIce',
    'playerPositionThatDidEvent', 'shotType', 'shotAngle',
    'shooterName', 'team', 'shotDistance'
]

# Restrict dataset to selected columns
shots = shots[keep_variables]

# Function to determine strength situation
def determine_strength(row):
    if row['homeSkatersOnIce'] > row['awaySkatersOnIce']:
        return 'Home Power Play'
    elif row['homeSkatersOnIce'] < row['awaySkatersOnIce']:
        return 'Away Power Play'
    else:
        return 'Even Strength'

# Apply the function to create new column
```

```
shots['strengthSituation'] = shots.apply(determine_strength, axis=1)

# Create a copy before dropping skater count columns
shots_k = shots.copy()

# Drop skater count columns
shots_k = shots_k.drop(columns=['homeSkatersOnIce', 'awaySkatersOnIce'])

# Clean Numeric Features
num_cols = [
    'arenaAdjustedShotDistance', 'arenaAdjustedXCord', 'arenaAdjustedYCord',
    'shotAngle', 'shotDistance', 'xGoal'
]
for col in num_cols:
    if col in shots_k.columns:
        shots_k[col] = pd.to_numeric(shots_k[col], errors='coerce')
        shots_k[col] = shots_k[col].fillna(shots_k[col].median())
# Clean Categorical Features
cat_cols = [
    'isHomeTeam', 'goalieNameForShot', 'homeTeamCode', 'lastEventCategory',
    'location', 'playerPositionThatDidEvent', 'shotType', 'shooterName', 'team', 'str
]
for col in cat_cols:
    if col in shots_k.columns:
        shots_k[col] = shots_k[col].fillna('Unknown')
# Convert booleans to int
if 'isHomeTeam' in shots_k.columns:
    shots_k['isHomeTeam'] = shots_k['isHomeTeam'].astype(int)

# Save file for analysis
shots_k.to_csv(r"/content/drive/MyDrive/Capstone/shots_cleaned.csv", index=False)
```

Appendix B: Exploratory Analysis

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
# Load your cleaned dataset
shots = pd.read_csv('/content/drive/MyDrive/Capstone/shots_cleaned.csv')

# Display descriptive statistics of numerical columns
display(shots.describe())
```

```
# Top ten shooters by goals and shooting pct
# Group by shooterName and calculate total shots and goals
shooter_stats = shots.groupby('shooterName').agg(
    total_shots=('goal', 'count'),
    total_goals=('goal', 'sum')
).reset_index()

# Calculate shooting percentage
shooter_stats['shooting_percentage'] = (shooter_stats['total_goals'] / shooter_stats[

# Sort by total goals and shooting percentage
top_goal_scorers = shooter_stats.sort_values(by=['total_goals', 'shooting_percentage']

print("Top 10 Shooters by Total Goals and Shooting Percentage:")
display(top_goal_scorers)

min_shots = 50 # minimum number of shots
top_percentage_shooters = shooter_stats[shooter_stats['total_shots'] >= min_shots].so

print(f"\nTop 10 Shooters by Shooting Percentage (minimum {min_shots} shots):")
display(top_percentage_shooters)

# Goals compared to non-goals with shot distance
min_dist = shots['shotDistance'].min()
max_dist = shots['shotDistance'].max()
# Create bins for every 5 feet
bins = np.arange(0, max_dist + 5, 5)

# Create a new column for binned shot distances
shots['shotDistance_bin'] = pd.cut(shots['shotDistance'], bins=bins, right=False)

# Group by binned shotDistance and goal, then count occurrences
binned_counts = shots.groupby(['shotDistance_bin', 'goal']).size().unstack(fill_value

# Rename columns for clarity (0: No Goal, 1: Goals)
binned_counts.rename(columns={0: 'No Goal', 1: 'Goals'}, inplace=True)

# Plot the stacked bar chart (histogram-like visualization)
plt.figure(figsize=(12, 6))
binned_counts[['Goals', 'No Goal']].plot(kind='bar', stacked=True, color=['orange', '

plt.title('Shot Distance Distribution by Goal Outcome (Histogram)')
plt.xlabel('Shot Distance (ft)')
plt.ylabel('Number of Shots')
plt.xticks(rotation=90) # Rotate x-axis labels for better readability
# Removed the grid lines as requested
# plt.grid(axis='y', linestyle='--', alpha=0.7)
```

```
plt.tight_layout()
plt.show()

# Goals compared to non-goals with shot-type
# Group by shooterName and calculate total shots and goals
shooter_stats = shots.groupby('shooterName').agg(
    total_shots=('goal', 'count'),
    total_goals=('goal', 'sum')
).reset_index()

# Calculate shooting percentage
shooter_stats['shooting_percentage'] = (shooter_stats['total_goals'] / shooter_stats[

# Sort by total goals and shooting percentage
top_goal_scorers = shooter_stats.sort_values(by=['total_goals', 'shooting_percentage']

print("Top 10 Shooters by Total Goals and Shooting Percentage:")
display(top_goal_scorers)

min_shots = 50 # minimum number of shots
top_percentage_shooters = shooter_stats[shooter_stats['total_shots'] >= min_shots].so

print(f"\nTop 10 Shooters by Shooting Percentage (minimum {min_shots} shots):")
display(top_percentage_shooters)

# Top 10 Goalies by Save Percentage
# Group by goalieNameForShot and calculate total shots faced
goalie_stats = shots.groupby('goalieNameForShot').agg(
    total_shots_faced=('goal', 'count'),
    goals_against=('goal', lambda x: (x == 1).sum()) # Count where 'goal' is 1
).reset_index()

# Calculate saves
goalie_stats['saves'] = goalie_stats['total_shots_faced'] - goalie_stats['goals_again

# Calculate save percentage
goalie_stats['save_percentage'] = (
    (goalie_stats['saves'] / goalie_stats['total_shots_faced']) * 100
).fillna(0)

# Filter for goalies with a minimum number of shots faced to ensure meaningful percen
min_shots_faced = 500 # Define a minimum number of shots for a goalie to be considere
filtered_goalie_stats = goalie_stats[goalie_stats['total_shots_faced'] >= min_shots_f

# Sort by save percentage in descending order
top_save_percentage_goalies = filtered_goalie_stats.sort_values(by='save_percentage',

print(f"Top 10 Goalies by Save Percentage (minimum {min_shots_faced} shots faced):")
```

```
display(top_save_percentage_goalies)

# Leon Draisaitl goal plot on hockey rink
# Filter the dataset to include only goals scored by Leon Draisaitl
draisaitl_goals = shots[(shots['shooterName'] == 'Leon Draisaitl') & (shots['goal'] =

# Display the first few rows and the shape of the draisaitl_goals DataFrame
print("First 5 rows of Leon Draisaitl's goals:")
display(draisaitl_goals.head())
print("\nShape of Leon Draisaitl's goals DataFrame:")
print(draisaitl_goals.shape)

# Code so shot type is colored on shot plot
# Get unique shot types from Leon Draisaitl's goals
unique_shot_types = draisaitl_goals['shotType'].unique()

# Create a color mapping for each shot type
cmap = plt.colormaps.get_cmap('tab10')
shot_type_colors = {shot_type: cmap(i) for i, shot_type in enumerate(unique_shot_type

# Print the color mapping
print("Color mapping for shot types:")
print(shot_type_colors)

# Create the shot plot for Draisaitl
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.patches as patches # Import patches for curved lines

# Define standard NHL rink dimensions (in feet)
rink_length = 200
rink_width = 85
center_ice_to_goal_line = rink_length / 2
blue_line_from_end_boards = 75 # Corrected based on user input
faceoff_circle_from_goal_line = 69
faceoff_circle_radius = 15
goal_line_from_end_boards = 11
goal_width = 6
goal_depth = 4 # Net depth
corner_radius = 28 # Standard NHL corner radius

# Define coordinates relative to center ice (0,0)
rink_left, rink_right = -rink_length / 2, rink_length / 2
rink_bottom, rink_top = -rink_width / 2, rink_width / 2
goal_line_left_x, goal_line_right_x = -center_ice_to_goal_line, center_ice_to_goal_li
blue_line_left_x = -rink_length / 2 + blue_line_from_end_boards # -100 + 75 = -25
blue_line_right_x = rink_length / 2 - blue_line_from_end_boards # 100 - 75 = 25
center_line_x = 0
```

```
goal_left_x, goal_right_x = -center_ice_to_goal_line, center_ice_to_goal_line
goal_left_post_y, goal_right_post_y = -goal_width / 2, goal_width / 2

# Faceoff circles (center coordinates relative to center ice)
faceoff_left_x, faceoff_right_x = -faceoff_circle_from_goal_line, faceoff_circle_from
faceoff_top_y, faceoff_bottom_y = rink_width / 4, -rink_width / 4
center_faceoff_x, center_faceoff_y = 0, 0


# Create a figure and axes for the plot
fig, ax = plt.subplots(figsize=(12, 6))

# Plot rink outline
straight_horizontal_length = rink_length - 2 * corner_radius
straight_vertical_length = rink_width - 2 * corner_radius

# Bottom straight
ax.plot([-rink_length/2 + corner_radius, rink_length/2 - corner_radius], [-rink_width
# Top straight
ax.plot([-rink_length/2 + corner_radius, rink_length/2 - corner_radius], [rink_width/
# Left straight
ax.plot([-rink_length/2, -rink_length/2], [-rink_width/2 + corner_radius, rink_width/
# Right straight
ax.plot([rink_length/2, rink_length/2], [-rink_width/2 + corner_radius, rink_width/2

# Curved corners
# Bottom-left
arc_bl = patches.Arc((-rink_length/2 + corner_radius, -rink_width/2 + corner_radius),
ax.add_patch(arc_bl)
# Bottom-right
arc_br = patches.Arc((rink_length/2 - corner_radius, -rink_width/2 + corner_radius),
ax.add_patch(arc_br)
# Top-left
arc_tl = patches.Arc((-rink_length/2 + corner_radius, rink_width/2 - corner_radius),
ax.add_patch(arc_tl)
# Top-right
arc_tr = patches.Arc((rink_length/2 - corner_radius, rink_width/2 - corner_radius), 2
ax.add_patch(arc_tr)


# Plot goal lines
ax.plot([goal_line_left_x, goal_line_left_x], [rink_bottom, rink_top], color='red', l
ax.plot([goal_line_right_x, goal_line_right_x], [rink_bottom, rink_top], color='red',

# Plot blue lines
ax.plot([blue_line_left_x, blue_line_left_x], [rink_bottom, rink_top], color='blue',
ax.plot([blue_line_right_x, blue_line_right_x], [rink_bottom, rink_top], color='blue'
```

```
# Plot center line
ax.plot([center_line_x, center_line_x], [rink_bottom, rink_top], color='red', linesty

# Plot center circle
center_circle = plt.Circle((center_faceoff_x, center_faceoff_y), faceoff_circle_radiu
ax.add_patch(center_circle)

# Plot faceoff circles (approximate locations)
faceoff_circle_left_top = plt.Circle((faceoff_left_x, faceoff_top_y), faceoff_circle_
faceoff_circle_left_bottom = plt.Circle((faceoff_left_x, faceoff_bottom_y), faceoff_c
faceoff_circle_right_top = plt.Circle((faceoff_right_x, faceoff_top_y), faceoff_circl
faceoff_circle_right_bottom = plt.Circle((faceoff_right_x, faceoff_bottom_y), faceoff

ax.add_patch(faceoff_circle_left_top)
ax.add_patch(faceoff_circle_left_bottom)
ax.add_patch(faceoff_circle_right_top)
ax.add_patch(faceoff_circle_right_bottom)


# Plot goals (nets) - Corrected placement
goal_left_net = plt.Rectangle((goal_left_x, goal_left_post_y), -goal_depth, goal_widt
goal_right_net = plt.Rectangle((goal_right_x, goal_left_post_y), goal_depth, goal_wid

ax.add_patch(goal_left_net)
ax.add_patch(goal_right_net)


# Plot Leon Draisaitl's goals, color-coded by shot type
for index, goal in draisaitl_goals.iterrows():
    # Ensure the shotType exists in the color mapping
    if goal['shotType'] in shot_type_colors:
        label = goal['shotType']
        if ax.get_legend() and label not in [l.get_text() for l in ax.get_legend().ge
            ax.scatter(goal['arenaAdjustedXCord'], goal['arenaAdjustedYCord'],
                       color=shot_type_colors[label], label=label)
        else:
            ax.scatter(goal['arenaAdjustedXCord'], goal['arenaAdjustedYCord'],
                       color=shot_type_colors[label])
    else:
        # Handle cases where shotType might not have a color mapping (e.g., if draisa
        if ax.get_legend() and 'Unknown Shot Type' not in [l.get_text() for l in ax.g
            ax.scatter(goal['arenaAdjustedXCord'], goal['arenaAdjustedYCord'], color=
        else:
            ax.scatter(goal['arenaAdjustedXCord'], goal['arenaAdjustedYCord'], color=

# Set plot title and labels
ax.set_title('Leon Draisaitl Goals Location by Shot Type')
ax.set_xlabel('X Coordinate (ft)')
```

```
ax.set_ylabel('Y Coordinate (ft)')

# Set axis limits to show the entire rink
ax.set_xlim([rink_left - 10, rink_right + 10])
ax.set_ylim([rink_bottom - 10, rink_top + 10])

# Add a legend
handles, labels = ax.get_legend_handles_labels()
by_label = dict(zip(labels, handles))
if by_label: # Only add legend if there are items to legend
    ax.legend(by_label.values(), by_label.keys(), title='Shot Type', bbox_to_anchor=(

ax.set_aspect('equal', adjustable='box')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

Appendix C: Feature Classification

```
# Separate target and features
X = shots.drop('goal', axis=1)
y = shots['goal']

# Encode categorical variables
cat_cols = X.select_dtypes(include=['object']).columns
le = LabelEncoder()

for col in cat_cols:
    X[col] = le.fit_transform(X[col].astype(str))

# Standardize numerical features
scaler = StandardScaler()
num_cols = X.select_dtypes(include=['int64', 'float64']).columns
X[num_cols] = scaler.fit_transform(X[num_cols])

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    stratify=y,
    random_state=42
)

# Logistic Regression Feature Classification

logreg = LogisticRegression(penalty='l1', solver='liblinear', max_iter=1000)
logreg.fit(X_train, y_train)
```

```
logreg_importances = pd.Series(abs(logreg.coef_[0]), index=X.columns).sort_values(asc

print("Top 10 features (Logistic Regression):")
print(logreg_importances.head(10))

# Random Forest Feature Classification

# Train a Random Forest Classifier
rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)

rf_importances = pd.Series(rf.feature_importances_, index=X.columns).sort_values(asce

# Plot top 10 features
plt.figure(figsize=(6,4))
rf_importances.plot(kind='barh')
plt.gca().invert_yaxis()
plt.title("Feature Importances (Random Forest)")
plt.xlabel("Importance")
plt.show()

# Decision Tree Feature Classification

import pandas as pd
import matplotlib.pyplot as plt

# Get feature importances from the trained Decision Tree model
dt_importances = pd.Series(dt_model.feature_importances_, index=X.columns).sort_value

# Plot top 10 features
plt.figure(figsize=(6, 4))
dt_importances.head(10).plot(kind='bar')
plt.title("Feature Importances (Decision Tree)")
plt.xlabel("Feature")
plt.ylabel("Importance")
plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for better readability
plt.tight_layout()
plt.show()
```

Appendix D: Model Testing

```
# Logistic Regression Model

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score,
```

```
model = LogisticRegression(class_weight='balanced', solver='liblinear', random_state=
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

# Calculate evaluation metrics
accuracy_logreg = accuracy_score(y_test, y_pred)
precision_logreg = precision_score(y_test, y_pred)
recall_logreg = recall_score(y_test, y_pred)
f1_logreg = f1_score(y_test, y_pred)

print("Logistic Regression Model Performance:")
print(f"Accuracy: {accuracy_logreg:.4f}")
print(f"Precision: {precision_logreg:.4f}")
print(f"Recall: {recall_logreg:.4f}")
print(f"F1-score: {f1_logreg:.4f}")

# print classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, digits=3))

# display confusion matrix
cm_logreg = confusion_matrix(y_test, y_pred)
print(cm_logreg)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_logreg)
disp.plot(cmap='Blues')
plt.figure(figsize=(3,2))
plt.title('Confusion Matrix (Logistic Regression)')
plt.show()

# XGBoost Model

%pip install xgboost
from xgboost import XGBClassifier

# Instantiate XGBClassifier
xgb_model = XGBClassifier(random_state=42)

# Train the XGBoost model
xgb_model.fit(X_train, y_train)

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,

# Make predictions on the test set
y_pred_xgb = xgb_model.predict(X_test)

# Calculate evaluation metrics
accuracy_xgb = accuracy_score(y_test, y_pred_xgb)
```

```
precision_xgb = precision_score(y_test, y_pred_xgb)
recall_xgb = recall_score(y_test, y_pred_xgb)
f1_xgb = f1_score(y_test, y_pred_xgb)

print("XGBoost Model Performance:")
print(f"Accuracy: {accuracy_xgb:.4f}")
print(f"Precision: {precision_xgb:.4f}")
print(f"Recall: {recall_xgb:.4f}")
print(f"F1-score: {f1_xgb:.4f}")

# print classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_xgb))

# display confusion matrix
cm_xgb = confusion_matrix(y_test, y_pred_xgb)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_xgb)
disp.plot(cmap='Blues')
plt.figure(figsize=(4,2))
plt.title('Confusion Matrix (XGBoost)')
plt.show()

# Decision Tree Model

from sklearn.tree import DecisionTreeClassifier

# Instantiate DecisionTreeClassifier
dt_model = DecisionTreeClassifier(random_state=42)

# Train the Decision Tree model
dt_model.fit(X_train, y_train)

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
import matplotlib.pyplot as plt
import pandas as pd

# Make predictions on the test set
y_pred_dt = dt_model.predict(X_test)

# Calculate evaluation metrics
accuracy_dt = accuracy_score(y_test, y_pred_dt)
precision_dt = precision_score(y_test, y_pred_dt)
recall_dt = recall_score(y_test, y_pred_dt)
f1_dt = f1_score(y_test, y_pred_dt)

print("Decision Tree Model Performance:")
print(f"Accuracy: {accuracy_dt:.4f}")
print(f"Precision: {precision_dt:.4f}")
```

```
print(f"Recall: {recall_dt:.4f}")
print(f"F1-score: {f1_dt:.4f}")

# print classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_dt))

# display confusion matrix
cm_dt = confusion_matrix(y_test, y_pred_dt)

disp = ConfusionMatrixDisplay(confusion_matrix=cm_dt)
disp.plot(cmap='Blues')
plt.title('Confusion Matrix (Decision Tree)')
plt.show()

# Random Forest Model

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
import matplotlib.pyplot as plt

# Make predictions on the test set
y_pred_rf = rf.predict(X_test)

# Calculate evaluation metrics
accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf)
recall_rf = recall_score(y_test, y_pred_rf)
f1_rf = f1_score(y_test, y_pred_rf)

print("Random Forest Model Performance:")
print(f"Accuracy: {accuracy_rf:.4f}")
print(f"Precision: {precision_rf:.4f}")
print(f"Recall: {recall_rf:.4f}")
print(f"F1-score: {f1_rf:.4f}")

# print classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf))

# display confusion matrix
cm_rf = confusion_matrix(y_test, y_pred_rf)
print(cm_rf)
print("\nConfusion Matrix:")
print(cm_rf)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_rf)
disp.plot(cmap='Blues')
plt.figure(figsize=(4,2))
plt.title('Confusion Matrix (Random Forest)')
```

```
plt.show()
```

Appendix E: Statistical Analysis

```
# ANOVA F-Test

from scipy.stats import f_oneway
import pandas as pd

# Identify numerical columns, excluding 'goal'
num_cols = shots.select_dtypes(include=np.number).columns.tolist()
if 'goal' in num_cols:
    num_cols.remove('goal')

# Initialize a dictionary to store ANOVA results
anova_results = {}

# Loop through each numerical feature, perform ANOVA, and store results
for col in num_cols:
    # Create two groups: goal (1) and no-goal (0)
    group_goal = shots[shots['goal'] == 1][col].dropna()
    group_no_goal = shots[shots['goal'] == 0][col].dropna()

    # Ensure both groups have data to perform the test
    if len(group_goal) > 1 and len(group_no_goal) > 1:
        f_stat, p_val = f_oneway(group_goal, group_no_goal)
        anova_results[col] = {'F-statistic': f_stat, 'p-value': p_val}
    else:
        anova_results[col] = {'F-statistic': np.nan, 'p-value': np.nan}

# Convert the results dictionary into a Pandas DataFrame
anova_df = pd.DataFrame.from_dict(anova_results, orient='index')

# Sort the DataFrame by p-value in ascending order
anova_df = anova_df.sort_values(by='p-value')

# Display the sorted DataFrame
print("ANOVA F-test Results (sorted by p-value):")
display(anova_df)

# Chi-squared Test

import numpy as np

# Identify categorical columns (object dtype)
categorical_features = shots.select_dtypes(include=['object']).columns.tolist()
```

```
# Exclude 'goal' if it's in the list (though 'goal' is typically numerical/binary)
if 'goal' in categorical_features:
    categorical_features.remove('goal')

print("Identified Categorical Features:")
print(categorical_features)

from scipy.stats import chi2_contingency
import pandas as pd

# Dictionary to store Chi-squared test results
chi2_results = {}

print("Performing Chi-squared tests for categorical features...")

for feature in categorical_features:
    # Create a contingency table
    contingency_table = pd.crosstab(shots[feature], shots['goal'])

    # Perform Chi-squared test
    if not contingency_table.empty and (contingency_table.sum(axis=0) > 0).all() and
        chi2, p_value, _, _ = chi2_contingency(contingency_table)
        chi2_results[feature] = {'Chi2 Statistic': chi2, 'p-value': p_value}
    else:
        chi2_results[feature] = {'Chi2 Statistic': np.nan, 'p-value': np.nan}
        print(f"Warning: Not enough data for Chi-squared test on feature '{feature}'.

# Convert results to a DataFrame for better display
chi2_df = pd.DataFrame.from_dict(chi2_results, orient='index')
chi2_df.index.name = 'Feature'
chi2_df = chi2_df.sort_values(by='p-value')

print("\nChi-squared Test Results (Sorted by p-value):")
display(chi2_df)
```

Appendix F: ROC-AUC and PR-AUC

```
# Logistic Regression

from sklearn.metrics import roc_curve, auc, precision_recall_curve, RocCurveDisplay, 
import matplotlib.pyplot as plt

# Get predicted probabilities for the positive class (class 1)
y_pred_proba = model.predict_proba(X_test)[:, 1]

# --- ROC Curve and ROC-AUC ---
```

```
# Calculate False Positive Rate (FPR), True Positive Rate (TPR), and thresholds
fpr, tpr, thresholds_roc = roc_curve(y_test, y_pred_proba)

# Calculate ROC-AUC score
roc_auc = auc(fpr, tpr)

print(f"ROC-AUC Score: {roc_auc:.4f}")

# Plot ROC curve
plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
roc_display = RocCurveDisplay(fpr=fpr, tpr=tpr, roc_auc=roc_auc, estimator_name='Logi
roc_display.plot(ax=plt.gca())
plt.title('ROC Curve - Logistic Regression')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.grid(True)

# --- Precision-Recall Curve and PR-AUC ---

# Calculate precision, recall, and thresholds
precision, recall, thresholds_pr = precision_recall_curve(y_test, y_pred_proba)

# Calculate Precision-Recall AUC score
pr_auc = auc(recall, precision)

print(f"Precision-Recall AUC Score: {pr_auc:.4f}")

# Plot Precision-Recall curve
plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
pr_display = PrecisionRecallDisplay(precision=precision, recall=recall, estimator_nam
pr_display.plot(ax=plt.gca())
plt.title('Precision-Recall Curve - Logistic Regression')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.grid(True)

plt.tight_layout()
plt.show()

# XGBoost

from sklearn.metrics import roc_curve, auc, precision_recall_curve, RocCurveDisplay, 
import matplotlib.pyplot as plt

# Get predicted probabilities for the positive class (class 1)
y_pred_proba_xgb = xgb_model.predict_proba(X_test)[:, 1]
```

```
# --- ROC Curve and ROC-AUC ---

# Calculate False Positive Rate (FPR), True Positive Rate (TPR), and thresholds
fpr_xgb, tpr_xgb, thresholds_roc_xgb = roc_curve(y_test, y_pred_proba_xgb)

# Calculate ROC-AUC score
roc_auc_xgb = auc(fpr_xgb, tpr_xgb)

# Print the calculated ROC-AUC score
print(f"XGBoost ROC-AUC Score: {roc_auc_xgb:.4f}")

# Create a new figure for plotting
plt.figure(figsize=(8, 4))

# Plot the ROC curve for the XGBoost model
plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
roc_display_xgb = RocCurveDisplay(fpr=fpr_xgb, tpr=tpr_xgb, roc_auc=roc_auc_xgb, esti
roc_display_xgb.plot(ax=plt.gca())

# Add a title to the ROC plot
plt.title('ROC Curve - XGBoost')

# Add labels to the x-axis and y-axis
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.grid(True)

# --- Precision-Recall Curve and PR-AUC ---

# Calculate precision, recall, and thresholds
precision_xgb, recall_xgb, thresholds_pr_xgb = precision_recall_curve(y_test, y_pred_

# Calculate Precision-Recall AUC score
pr_auc_xgb = auc(recall_xgb, precision_xgb)

# Print the calculated Precision-Recall AUC score
print(f"XGBoost Precision-Recall AUC Score: {pr_auc_xgb:.4f}")

# Plot the Precision-Recall curve for the XGBoost model
plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
pr_display_xgb = PrecisionRecallDisplay(precision=precision_xgb, recall=recall_xgb, e
pr_display_xgb.plot(ax=plt.gca())
# Add a title to the Precision-Recall plot
plt.title('Precision-Recall Curve - XGBoost')
# Add labels to the x-axis and y-axis
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.grid(True)
```

```
# Ensure plots are properly laid out and display them
plt.tight_layout()
plt.show()

# Decision Tree

from sklearn.metrics import roc_curve, auc, precision_recall_curve, RocCurveDisplay, 
import matplotlib.pyplot as plt

# Get predicted probabilities for the positive class (class 1)
y_pred_proba_dt = dt_model.predict_proba(X_test)[:, 1]

# --- ROC Curve and ROC-AUC ---

# Calculate False Positive Rate (FPR), True Positive Rate (TPR), and thresholds
fpr_dt, tpr_dt, thresholds_roc_dt = roc_curve(y_test, y_pred_proba_dt)

# Calculate ROC-AUC score
roc_auc_dt = auc(fpr_dt, tpr_dt)

print(f"Decision Tree ROC-AUC Score: {roc_auc_dt:.4f}")

# Create a new figure for plotting with two subplots
plt.figure(figsize=(12, 6))

# Plot the ROC curve for the Decision Tree model
plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
roc_display_dt = RocCurveDisplay(fpr=fpr_dt, tpr=tpr_dt, roc_auc=roc_auc_dt, estimato
roc_display_dt.plot(ax=plt.gca())
plt.title('ROC Curve - Decision Tree')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.grid(True)

# --- Precision-Recall Curve and PR-AUC ---

# Calculate precision, recall, and thresholds
precision_dt, recall_dt, thresholds_pr_dt = precision_recall_curve(y_test, y_pred_pro

# Calculate Precision-Recall AUC score
pr_auc_dt = auc(recall_dt, precision_dt)

print(f"Decision Tree Precision-Recall AUC Score: {pr_auc_dt:.4f}")

# Plot the Precision-Recall curve for the Decision Tree model
plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
pr_display_dt = PrecisionRecallDisplay(precision=precision_dt, recall=recall_dt, esti
```

```
pr_display_dt.plot(ax=plt.gca())
plt.title('Precision-Recall Curve - Decision Tree')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.grid(True)

# Ensure plots are properly laid out and display them
plt.tight_layout()
plt.show()

# Random Forest

from sklearn.metrics import roc_curve, auc, precision_recall_curve, RocCurveDisplay, 
import matplotlib.pyplot as plt

# Get predicted probabilities for the positive class (class 1)
y_pred_proba_rf = rf.predict_proba(X_test)[:, 1]

# --- ROC Curve and ROC-AUC ---

# Calculate False Positive Rate (FPR), True Positive Rate (TPR), and thresholds
fpr_rf, tpr_rf, thresholds_roc_rf = roc_curve(y_test, y_pred_proba_rf)

# Calculate ROC-AUC score
roc_auc_rf = auc(fpr_rf, tpr_rf)

# Print the calculated ROC-AUC score
print(f"Random Forest ROC-AUC Score: {roc_auc_rf:.4f}")

# Create a new figure for plotting with two subplots
plt.figure(figsize=(12, 6))

# Plot the ROC curve for the Random Forest model
plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
roc_display_rf = RocCurveDisplay(fpr=fpr_rf, tpr=tpr_rf, roc_auc=roc_auc_rf, estimato
roc_display_rf.plot(ax=plt.gca())
plt.title('ROC Curve - Random Forest')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.grid(True)

# --- Precision-Recall Curve and PR-AUC ---

# Calculate precision, recall, and thresholds
precision_rf, recall_rf, thresholds_pr_rf = precision_recall_curve(y_test, y_pred_pro

# Calculate Precision-Recall AUC score
pr_auc_rf = auc(recall_rf, precision_rf)
```

```
# Print the calculated Precision-Recall AUC score
print(f"Random Forest Precision-Recall AUC Score: {pr_auc_rf:.4f}")

# Plot the Precision-Recall curve for the Random Forest model
plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
pr_display_rf = PrecisionRecallDisplay(precision=precision_rf, recall=recall_rf, esti
pr_display_rf.plot(ax=plt.gca())
plt.title('Precision-Recall Curve - Random Forest')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.grid(True)

# Ensure plots are properly laid out and display them
plt.tight_layout()
plt.show()
```